DiffAct++: Diffusion Action Segmentation

Daochang Liu, Qiyue Li, Anh-Dung Dinh, Tingting Jiang, Mubarak Shah, *Fellow, IEEE*, and Chang Xu, *Senior Member, IEEE*

***Abstract*—Understanding long-form videos requires precise temporal action segmentation. While existing studies typically employ multi-stage models that follow an iterative refinement process, we present a novel framework based on the denoising diffusion model that retains this core iterative principle. Within this framework, the model iteratively produces action predictions starting with random noise, conditioned on the features of the input video. To effectively capture three key characteristics of human actions, namely the position prior, the boundary ambiguity, and the relational dependency, we propose a cohesive masking strategy for the conditioning features. Moreover, a consistency gradient guidance technique is proposed, which maximizes the similarity between outputs with or without the masking, thereby enriching conditional information during the inference process. Extensive experiments are performed on four datasets, i.e., GTEA, 50Salads, Breakfast, and Assembly101. The results indicate that our proposed method outperforms or is on par with existing state-of-the-art techniques, underscoring the potential of generative approaches for action segmentation.**

***Index Terms*—Computer vision, diffusion models, human activity recognition.**

I. Introduction

TEMPORAL action segmentation has a pivotal role in comprehending human behaviors in complex videos, which has manifold applications spanning from video surveillance [1], video summarization [2] to skill assessment [3]. Essentially, temporal action segmentation aims to process a long video and produce a sequence that delineates the action category for each frame.

Recent years have witnessed notable advancements in this domain due to the development of multi-stage models [4], [5], [6], [7]. These models operate by layering multiple stages where the initial prediction is made by the first stage, and subsequent stages refine that prediction based on previous outputs. Numerous architectures have been investigated to realize this multi-stage concept, including the MS-TCN [4], [5] that uses dilated temporal convolutional layers and the ASFormer [7] that incorporates attention mechanisms. The effectiveness of these multi-stage models can be mainly credited to the iterative refinement paradigm they adopt, which adeptly captures the temporal dynamics of actions and considerably alleviates over-segmentation mistakes [8].

In this study, we introduce a temporal action segmentation method that adheres to the principle of iterative refinement, yet leverages a fundamentally new generative approach by integrating the denoising diffusion model. The diffusion model [9], [10], [11], [12], known for its straightforward training protocol and superior generation capability, has quickly risen as a prominent class of generative models. Within the diffusion model, a forward process injects noise into the data incrementally, while a complementary reverse process gradually eliminates the noise and ultimately allows for the generation of new samples from the data distribution commencing from pure random noise. The iterative denoising approach used in the reverse process naturally aligns with the iterative refinement idea for action segmentation. This prompts us to reconsider action segmentation from a generative perspective, using the diffusion model illustrated in Fig. 1, where action segmentation can be framed as the generation of an action sequence conditioned on the given video input. A notable benefit of such diffusion-based action segmentation is its ability not only to discriminatively map video frames to specific actions, but also to implicitly grasp the intrinsic prior knowledge of human actions using generative modeling. This prior modeling can be further explicitly associated with inductive biases in accordance with three key characteristics of human behavior. The first characteristic is the temporal *position prior*, which implies that certain actions are predisposed to occur at specific times within a video. For example, in a video depicting salad preparation, actions like cutting vegetables typically manifest in the middle, while serving the salad onto the plate is mostly seen towards the end. The second characteristic pertains to the *boundary prior*, indicating that the shifts between actions occur smoothly, resulting in visual features being ambiguous near the boundaries to be correctly classified. The third characteristic is the *relation*

Received 6 November 2023; revised 16 November 2024; accepted 24 November 2024. Date of publication 29 November 2024; date of current version 5 February 2025. This work was supported in part by Australian Research Council under Grant DP210101859 and Grant FT230100549, and in part by the Natural Science Foundation of China under Grant 62088102. Recommended for acceptance by J.C. Niebles. *(Corresponding author: Chang Xu.)*

Daochang Liu is with the School of Physics, Mathematics and Computing, University of Western Australia, Crawley, WA 6009, Australia (e-mail: daochang.liu@uwa.edu.au).

Qiyue Li is with the National Engineering Research Center of Visual Technology, National Key Laboratory for Multimedia Information Processing, School of Computer Science, National Biomedical Imaging Center, Peking University, Beijing 100871, China, and also with the School of Mathematical Sciences, Peking University, Beijing 100871, China (e-mail: liqiyue@pku.edu.cn).

Anh-Dung Dinh and Chang Xu are with the School of Computer Science, Faculty of Engineering, University of Sydney, Darlington, NSW 2008, Australia (e-mail: adin6536@uni.sydney.edu.au; c.xu@sydney.edu.au).

Tingting Jiang is with the National Engineering Research Center of Visual Technology, National Key Laboratory for Multimedia Information Processing, School of Computer Science, National Biomedical Imaging Center, Peking University, Beijing 100871, China (e-mail: ttjiang@pku.edu.cn).

Mubarak Shah is with the Center for Research in Computer Vision, University of Central Florida, Orlando, FL 32816-2365 USA (e-mail: shah@crcv.ucf.edu).

This article has supplementary downloadable material available at https://doi.org/10.1109/TPAMI.2024.3509434, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2024.3509434

0162-8828 © 2024 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.

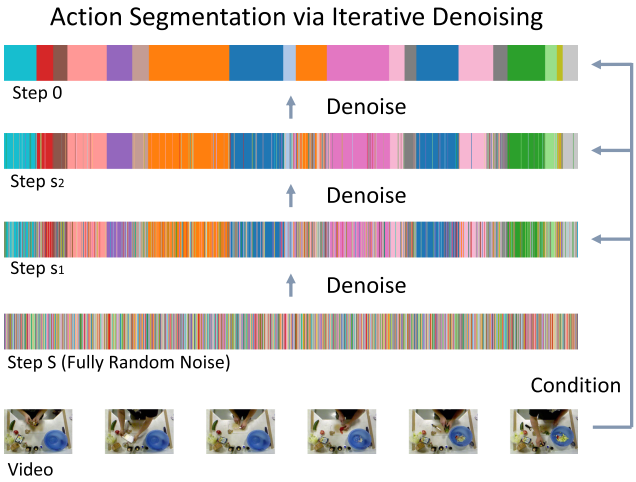


Fig. 1. Our method formulates action segmentation as a frame-wise action sequence generation problem conditioned on the video features. The prediction is obtained via an iterative denoising process at the inference stage. Colors in the barcodes represent different actions.

prior, which signifies that human actions generally conform to an inherent temporal arrangement. For instance, the act of cutting a cucumber typically comes after peeling a cucumber. This temporal relationship is different from the position prior as it emphasizes the order in comparison to other actions, whereas the position prior focuses on the absolute position on the timeline. To collectively leverage these patterns of human behaviors, we design a condition masking strategy that seamlessly integrates into our newly proposed framework.

In our approach, we conceptualize temporal action segmentation as a conditional generation task, wherein the frame-wise action label sequence is generated conditioned on the input video. In the training phase, the model receives both the input video features and a distorted action segmentation sequence, which is derived from the ground truth with varying amounts of added noise. The model is then trained to remove this noise and recover the actual ground truth sequence. To achieve this, we impose three loss functions to reconstruct the ground truth sequence, comprising a standard cross-entropy loss for action classification, a temporal smoothness loss as regularization, and an alignment loss for additional supervision on boundaries. During the inference phase, the model iteratively refines a random sequence in line with the reversed diffusion process, ultimately producing the predicted action sequence. On the other hand, to boost the modeling of the three previously mentioned priors related to human actions, our approach utilizes a condition masking method during the training phase, which encourages the model to consider information beyond visual cues, such as temporal positions, duration of actions, and the surrounding temporal context. The masked conditions transform the generative learning of action sequences from a solely conditional approach to a blend of fully conditional, partially conditional, and unconditional ones to strengthen the three action priors at the same time.

An earlier version of this manuscript appeared previously as a conference paper [13]. The current work makes substantial improvements in terms of methodology and experiments. First, we extend our method with sampling guidance to augment the model during the inference process without retraining. We specifically propose a new guidance technique, termed consistency gradient guidance, which is synergic with our condition masking. This enhancement reinforces the conditional information by compelling the unconditional model to yield results similar to the conditional model for intermediate inference steps. A gradient term is integrated into the iterative update rule at inference, which maximizes the consistency between model outputs with masked and unmasked conditions. Second, we incorporate several implementation strategies to improve the effectiveness and efficiency of the proposed consistency gradient guidance in practice. Third, we add extended experiments that qualitatively and quantitatively evaluate both our diffusion-based action segmentation framework and the consistency gradient guidance.

The following part of the paper is organized as below. In Section II, we provide a review of related works that include both temporal action segmentation methods and diffusion models. In Section III, we lay the foundation of diffusion and denoising processes as prerequisites for our proposed method. In Section IV, we describe our approach, including the training process and objectives, action prior modeling, and the inference procedure with and without consistency gradient guidance. In Section V, we detail the experimental settings and present the corresponding quantitative and qualitative results. In Section VI, we further discuss the prior modeling of our method, as well as the limitations and future directions. In Section VII, we conclude our paper and summarize our contributions.

II. Related Works

A. Temporal Action Segmentation

Temporal action segmentation [8] represents a challenging task in video understanding, in which input videos can last several minutes. To capture the long-range dependencies between actions, a rich variety of temporal models have been explored in past research. These range from initial approaches using recurrent neural networks [14], [15], advancing to temporal convolutional networks [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], graph neural networks [29], [30], [31], [32], and more recently, transformers [33], [34], [35], [36], [37], [38], [39], [40]. Particularly noteworthy are multi-stage models [4], [5], [6], [7], [25], [33], [41] that refine results incrementally, using the outcome from one stage to inform the subsequent stage. These models excel in understanding the temporal context and reducing errors due to over-segmentation [8]. MS-TCN [4] is a milestone work that introduces the multi-stage architecture to the action segmentation task where each stage is composed of dilated convolutions with increasing dilation rates, which is further improved to be MS-TCN++ [5] later by decoupling the first prediction stage and the following refinement stages. C2F-TCN [42] ensembles the outputs from multiple decoder layers to afford a "coarse-to-fine" fusion of predictions at different temporal resolutions. ASFormer [7] is a seminal transformer model that exploits the attention mechanism within a hierarchical window to efficiently process long sequences. Recently,

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.

ASPnet [43] proposes a multimodal approach with disentangled feature fusion for temporal action segmentation, and UnLoc [44] proposes a unified framework based on video-text fusion for both temporal action segmentation and localization. In addition to architecture designs, another line of work focuses on acquiring more accurate and robust features through representation learning [45], [46], [47], [48], domain adaptation [49], [50], [51], or data augmentation [52].

In the domain of temporal action segmentation, the position prior, the boundary ambiguity, and the relation prior are three beneficial inductive biases. While the boundary ambiguity and the relation prior have garnered some attention from the community, the position prior remains comparatively underexplored. To address the challenges posed by boundary ambiguity, the boundary-aware cascade network [6] introduces a local barrier pooling mechanism to facilitate the assignment of adaptive aggregation weights on the boundaries. Ishikawa et al. integrate a complementary network branch to regress boundary locations [53], and Chen et al. estimate uncertainty arising from unclear boundaries by Monte-Carlo sampling [54]. An alternative approach involves smoothing the annotation into a soft version, reducing the likelihood of actions around the boundaries [55]. In terms of the relation prior, DTL [56] extracts relational constraints from the annotations and applies these rules through a temporal logic loss in the training phase. Br-Prompt [47] resorts to multimodal learning by using text prompts to supervise contextual relationships between consecutive actions. Meanwhile, GTRM [29] and UVAST [57] employ graph convolutional networks and sequence-to-sequence translation techniques, respectively, to improve relational reasoning at the segment level. Subsequently, it has become a recent trend to build separate modules or pathways for frames and segments [58], [59], [60], [61] to grasp temporal correlations not only between frames but also between segments. In this paper, we simply use condition masking to address position prior, boundary ambiguity, and relation prior simultaneously, without the need for specific designs for each aspect.

A related work [62] also employs generative learning for action segmentation. This method utilizes GANs to synthesize intermediate action codes for each frame to assist recognition. In contrast, our method generates action label sequences via the diffusion model. We also notice a concurrent work [63] using diffusion models for action segmentation, which employs the Fourier transform to capture long-term dependencies. Beyond the most studied single-label and single-view setting, another body of works [64], [65], [66] address multi-label or multi-view action segmentation on challenging datasets such as TSU [67] and Charades [68]. Apart from the fully supervised temporal action segmentation, this task can also be studied under weakly supervised [69], semi-supervised [70], or unsupervised [71] settings. We refer the readers to a recent survey [8] for a comprehensive review.

B. Diffusion Models

The diffusion models [9], [10], [12], [72], which have been theoretically unified with the score-based models [73], [74], [75], are recognized for their stable training processes and the eliminated need for adversarial mechanisms in generative learning. Diffusion-based generation has achieved impressive results in various areas, including image generation [11], [76], [77], natural language generation [78], text-to-image synthesis [79], [80], audio generation [81], [82] and so on. There is also a rich body of work on video generation or video editing using diffusion models [83], [84], [85], [86]. For example, PYoCo [86] replaces the i.i.d. noise in diffusion models with temporally correlated noise to maintain coherence among the video frames generated. Diffusion models have been successfully repurposed for image understanding tasks in computer vision very recently, such as object detection [87], image segmentation [88], [89], and pose estimation [90], [91]. However, only very few works targeted video understanding tasks. A pre-trained diffusion model was fine-tuned to predict video memorability [92], and a frequency-aware diffusion module was proposed for video captioning [93]. Some concurrent works explore diffusion models for temporal action localization [94], video moment retrieval [95], and video anomaly detection [96], [97], [98]. We refer the reader to this survey on diffusion models in vision [9] for a thorough review. In this paper, we recognize that diffusion models, given their iterative refinement properties, are especially suitable for temporal action segmentation. In pose estimation [90], [91], diffusion models have been used to handle the high indeterminacy due to depth ambiguity and occlusions. Such indeterminacy also exists in action segmentation, such as around ambiguous action boundaries. Our generative diffusion model helps accommodate the action ambiguity along with the prior modeling.

III. Preliminaries

We begin by familiarizing the reader with the foundational concepts of diffusion models. Diffusion models [10], [12] are designed to approximate the data distribution $q(\mathrm{x}_0)$ with a model distribution $p_\theta(\mathrm{x}_0)$. The *forward process* or *diffusion process* perturbs the real data $\mathrm{x}_0 \sim q(\mathrm{x}_0)$ into a series of noisy data $\mathrm{x}_1, \mathrm{x}_2, \ldots, \mathrm{x}_S$. Conversely, the *reverse process* or *denoising process* gradually removes noise, transitioning from pure noise $\mathrm{x}_S \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ to $\mathrm{x}_{S-1}, \mathrm{x}_{S-2}, \ldots$ until the desired $\mathrm{x}_0 \sim p_\theta(\mathrm{x}_0)$ is achieved. The entire procedure has S steps.

Forward Process: Formally, the forward process follows a predefined variance schedule $\beta_1, \beta_2, \ldots, \beta_S$ to add Gaussian noise into the data at each step in a Markovian manner: $q(\mathrm{x}_s|\mathrm{x}_{s-1}) = \mathcal{N}(\mathrm{x}_s; \sqrt{1-\beta_s}\mathrm{x}_{s-1}, \beta_s \boldsymbol{I})$. By denoting $\alpha_s = 1 - \beta_s$ and $\bar{\alpha}_s = \prod_{i=1}^{s} \alpha_i$, it is possible to directly derive x_s from x_0 in a closed form without recursion: $q(\mathrm{x}_s|\mathrm{x}_0) = \mathcal{N}(\mathrm{x}_s; \sqrt{\bar{\alpha}_s}\mathrm{x}_0, (1-\bar{\alpha}_s)\boldsymbol{I})$, which can be further simplified using the reparameterization trick:

$$\mathrm{x}_s = \sqrt{\bar{\alpha}_s}\mathrm{x}_0 + \epsilon\sqrt{1-\bar{\alpha}_s}. \tag{1}$$

The noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ is sampled from a normal distribution at each step and its magnitude is determined by $\sqrt{1-\bar{\alpha}_s}$ in (1).

Reverse Process: The reverse process initiates from x_S and incrementally reduces the noise to restore x_0. A single step in

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.

this process is denoted as $p_\theta(\mathrm{x}_{s-1}|\mathrm{x}_s)$:

$$p_\theta\left(\mathrm{x}_{s-1}|\mathrm{x}_s\right) = \mathcal{N}\left(\mathrm{x}_{s-1}; \boldsymbol{\mu}_\theta(\mathrm{x}_s, s), \sigma_s^2 \boldsymbol{I}\right), \tag{2}$$

where σ_s^2 is controlled by β_s, and $\boldsymbol{\mu}_\theta(\mathrm{x}_s, s)$ represents a predicted mean parameterized by a step-dependent neural network.

There are multiple approaches [99] to parameterize p_θ, including predicting the mean as in (2), predicting the noise ϵ, and predicting the raw data x_0. Our method uses the prediction of x_0. In this case, rather than directly estimating $\boldsymbol{\mu}_\theta(\mathrm{x}_s, s)$ as in (2), the model predicts x_0 via a neural network $f_\theta(\mathrm{x}_s, s)$. For model optimization, a mean squared error loss $\mathcal{L} = ||f_\theta(\mathrm{x}_s, s) - \mathrm{x}_0||^2$ can be employed to match $f_\theta(\mathrm{x}_s, s)$ with x_0. Each training iteration selects a random step $s \in_R \{1, 2, \ldots, S\}$.

During the inference phase, beginning with a pure noise $\mathrm{x}_S \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, the model can progressively lower the noise intensity according to the following update formula [10] using the trained network f_θ:

$$\mathrm{x}_{s-1} = \sqrt{\bar{\alpha}_{s-1}} f_\theta(\mathrm{x}_s, s) + \sqrt{1 - \bar{\alpha}_{s-1} - \sigma_s^2} \frac{\mathrm{x}_s - \sqrt{\bar{\alpha}_s} f_\theta(\mathrm{x}_s, s)}{\sqrt{1 - \bar{\alpha}_s}} + \sigma_s \epsilon. \tag{3}$$

By repeatedly applying (3), a new sample x_0 can be drawn from the model distribution p_θ along a trajectory $\mathrm{x}_S, \mathrm{x}_{S-1}, \ldots, \mathrm{x}_0$. It is worth noting that some advanced sampling strategies [10] omit certain steps in the trajectory, i.e., using a sparse trajectory $\mathrm{x}_S, \mathrm{x}_{S-\Delta}, \ldots, \mathrm{x}_0$ to improve sampling efficiency.

Incorporating conditional information into the diffusion model allows for better regulation of the generation procedure. The conditional model can be written as $f_\theta(\mathrm{x}_s, s, \mathbb{C})$, where the conditional information, symbolized as $\mathbb{C}$, serves as an additional input to the model. In the literature, the prevalent forms of conditional information encompass class labels [11], text prompts [79], [80], and image constraints [100].

IV. Method

We approach temporal action segmentation by casting it as a conditional generation task of temporal action sequences. For a video consisting of L frames with input video features $F \in \mathbb{R}^{L \times D}$ of D dimensions, we aim to approximate the data distribution of the ground truth action sequence $Y_0 \in \{0, 1\}^{L \times C}$, represented in one-hot encoding across C action classes. As the input features F are usually extracted per short clip by general-purpose pre-trained models, we utilize an encoder h_ϕ to enrich the features with long-range temporal information and tailor them for our specific task, i.e., $E = h_\phi(F)$. The resultant encoded feature is denoted as $E \in \mathbb{R}^{L \times D'}$ with D' dimensions.

We name our method *DiffAct* when it employs the vanilla inference process in Section IV-A3 and *DiffAct++* when it is combined with the consistency gradient guidance in Section IV-C.

A. Diffusion Action Segmentation

Our method establishes a diffusion process $Y_0, Y_1, \ldots, Y_S$ during training that converts the action ground truth Y_0 to almost pure noise Y_S. On the other hand, it also builds a denoising process $\hat{Y}_S, \hat{Y}_{S-1}, \ldots, \hat{Y}_0$ during inference that inverts a pure noise $\hat{Y}_S$ into the action prediction $\hat{Y}_0$.

1) Training: We train the model to reconstruct the ground truth action sequence from its perturbed versions so that it can learn the underlying distribution of human actions, as illustrated in Fig. 2. At each training iteration, a random diffusion step $s \in_R \{1, 2, \ldots, S\}$ is selected. Subsequently, a noise ϵ is drawn from a Gaussian distribution based on the accumulative noise schedule described in (1), which is then injected into the ground truth sequence Y_0 to obtain a corrupted sequence $Y_s \in [0, 1]^{L \times C}$:

$$Y_s = \sqrt{\bar{\alpha}_s} Y_0 + \epsilon \sqrt{1 - \bar{\alpha}_s}. \tag{4}$$

Given the corrupted sequence, we design a decoder g_ψ to denoise it:

$$P_s = g_\psi(Y_s, s, E \odot M), \tag{5}$$

where the output denoised sequence $P_s \in [0, 1]^{L \times C}$ represents action probabilities for each individual frame. Besides the primary input Y_s, the decoder also takes two additional inputs, which are step s and encoded video features E. Incorporating the step s as input enables the model to be aware of the intensities of the noise, facilitating the use of a single model across varying levels of corruption. Using encoded video features E as conditions, we ensure that the resulting action sequences are not just generically plausible, but also align with the given video. More importantly, conditional information is further diversified by element-wisely multiplying the feature E with a mask M to explicitly capture the three characteristics of human actions, which we will elaborate on later. The instantiation of both the decoder g_ψ and the encoder h_ϕ is flexible, where common network architectures for action segmentation can be used.

2) Loss Functions: After obtaining the denoised sequence P_s, we apply three loss functions as below to match it with the original ground truth Y_0.

Cross-Entropy Loss: The first one is a cross-entropy loss. This standard classification loss aims at minimizing the negative log-likelihood of the ground truth action class for each frame:

$$\mathcal{L}_s^{\text{ce}} = \frac{1}{LC} \sum_{i=1}^{L} \sum_{c=1}^{C} -Y_{0,i,c} \log P_{s,i,c}, \tag{6}$$

where i and c denote the frame index and the class index respectively.

Temporal Smoothness Loss: To encourage local consistency across time, the second loss is defined as the mean squared error between the log-likelihoods of consecutive video frames [4], [5]:

$$\mathcal{L}_s^{\text{smo}} = \frac{1}{(L-1)C} \sum_{i=1}^{L-1} \sum_{c=1}^{C} \left(\log P_{s,i,c} - \log P_{s,i+1,c}\right)^2. \tag{7}$$

Note that this temporal smoothness loss $\mathcal{L}_s^{\text{smo}}$ is bounded as in [4] to prevent extreme values.

Boundary Alignment Loss: Precise boundary detection is crucial for the temporal action segmentation task. Thus, the third loss focuses on aligning the action boundaries in the denoised sequence P_s and the ground truth sequence Y_0. To achieve this, we need to calculate the boundary probabilities from both P_s and Y_0. First, we derive a ground truth boundary

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.

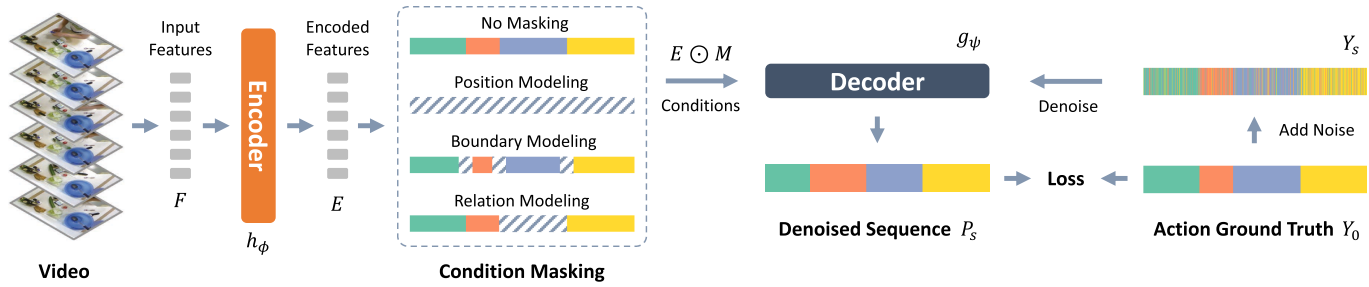


Fig. 2. Training overview. The proposed model is trained to reconstruct the clean action sequence from a corrupted sequence. Our condition masking technique enhances action prior modeling by selectively blocking specific features during the training phase. Areas with stripes are masked. Within the action sequences, each color corresponds to a distinct action. Input video features are pre-extracted.

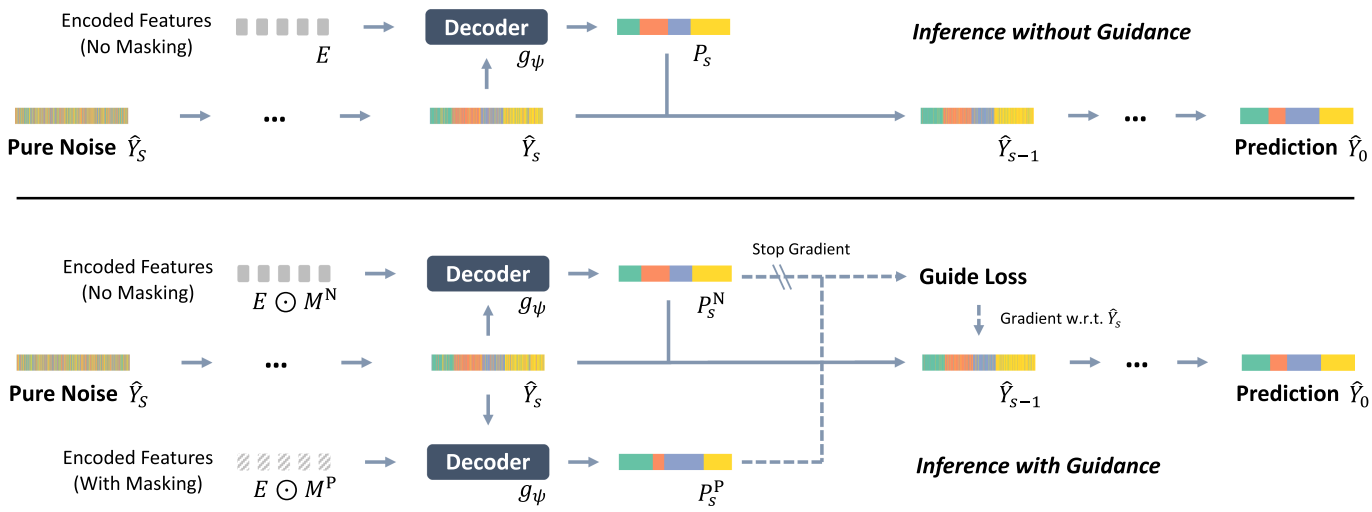


Fig. 3. Inference overview. At inference, the model begins with a random noise sequence and obtains results via an iterative denoising process. Top: The inference process of *DiffAct*. Bottom: The inference process of *DiffAct++* with consistency gradient guidance.

sequence $B \in \{0,1\}^{L-1}$ from the action ground truth Y_0, where $B_i = \mathbb{1}(Y_{0,i} \neq Y_{0,i+1})$. Given that action transitions often occur in a gradual manner, we smooth this sequence using a Gaussian filter to produce a soft version $\bar{B} = \lambda(B)$, where λ represents the Gaussian smoothing operation. For the boundaries in the denoised sequence, their probabilities are determined by taking the dot product of the action probabilities from adjacent frames in P_s, i.e., $1 - P_{s,i} \cdot P_{s,i+1}$. In the following, the boundaries derived from the ground truth sequence and the denoised sequence are then aligned using a binary cross-entropy loss, where $\bar{B}$ serves as the target:

$$\mathcal{L}_s^{\mathrm{bd}} = \frac{1}{L-1} \sum_{i=1}^{L-1} \big[-\bar{B}_i \log\left(1 - P_{s,i} \cdot P_{s,i+1}\right) - \left(1 - \bar{B}_i\right) \log\left(P_{s,i} \cdot P_{s,i+1}\right) \big]. \quad (8)$$

The above three loss components are computed at a randomly chosen diffusion step and are then mixed to be the final training loss:

$$\mathcal{L}^{\mathrm{sum}} = \mathcal{L}_s^{\mathrm{ce}} + \mathcal{L}_s^{\mathrm{smo}} + \mathcal{L}_s^{\mathrm{bd}}, s \in_R \{1, 2, \ldots, S\}. \quad (9)$$

The step s is uniformly sampled from $\{1, 2, \ldots, S\}$. With sufficient training iterations, the loss can cover all different steps and the model is trained to denoise at all noise intensities.

3) Inference: We first introduce the basic inference process of *DiffAct* in this section (Fig. 3 Top), which will be extended later in Section IV-C. Intuitively, the denoising decoder g_ψ is optimized to work with sequences having varying degrees of noise, even if the sequence is entirely composed of random noise. Consequently, during inference, the denoising decoder is able to begin with an entirely noisy sequence $\hat{Y}_S \in \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ and gradually decreases the noise level. Specifically, the update rule for each step is adapted from (3) in the following:

$$\hat{Y}_{s-1} = \sqrt{\bar{\alpha}_{s-1}} P_s + \sqrt{1 - \bar{\alpha}_{s-1} - \sigma_s^2} \frac{\hat{Y}_s - \sqrt{\bar{\alpha}_s} P_s}{\sqrt{1 - \bar{\alpha}_s}} + \sigma_s \epsilon, \quad (10)$$

where the $\hat{Y}_{s-1}$ is input into the decoder in the next step to produce the subsequent P_{s-1} so that the action sequence is iteratively refined. This step-by-step refinement yields a series of action sequences $\hat{Y}_S, \hat{Y}_{S-1}, \ldots, \hat{Y}_0$, culminating in the $\hat{Y}_0$ at the end that can closely approximate the underlying ground truth and is considered as the final prediction. To accelerate

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.

the inference speed, our method utilizes a sampling trajectory $\hat{Y}_S, \hat{Y}_{S-\Delta}, \ldots, \hat{Y}_0$ that skips equally spaced steps as in [10]. For this basic version of the inference process, the encoded features E are fed into the decoder without masking. We also fix the random seed for the denoising process to ensure determinism in our experiments.

B. Action Prior Modeling

Human behavior exhibits common patterns. In the context of action segmentation, we identify the position prior, the boundary ambiguity, and the relation prior as three key pieces of the prior knowledge. A distinct benefit of using diffusion-based action segmentation is its ability to capture the prior distribution of action sequences through its generative learning capacity. This enables us to further exploit the three action priors by controlling the conditional information for the model.

In detail, we develop a condition masking strategy to regulate the conditional information by applying a mask $M \in \{0,1\}^L$ on the conditioning features E used in (5). During each training iteration, the mask is randomly sampled from a predefined set, $M \in_R \{M^{\mathtt{N}}, M^{\mathtt{P}}, M^{\mathtt{B}}, M^{\mathtt{R}}\}$, the specific elements of which are elaborated below.

No Masking (N)*:* The first type is a basic all-one mask $M^{\mathtt{N}} = \mathbf{1}$ that allows the features of all frames to pass into the decoder. This simple mask offers full conditional information for the model to facilitate the mapping of visual features to action classes.

Masking for Position Prior (P)*:* The second type employs an all-zero mask $M^{\mathtt{P}} = \mathbf{0}$, which completely nullifies visual features. In the absence of visual features, the decoder becomes dependent only on cues such as video length and frame positions within the video. Consequently, this encourages the modeling of the positional prior.

Masking for Boundary Prior (B)*:* The inherent uncertainty in action transitions means that visual features near transition boundaries might be unreliable. Hence, the third mask $M^{\mathtt{B}}$ removes the features of the frames close to the boundaries based on the soft ground truth $\bar{B}$. This mask is defined as $M_i^{\mathtt{B}} = \mathbb{1}(\bar{B}_i < 0.5), i \in \{1,2,\ldots,L\}$. This mask $M^{\mathtt{B}}$ encourages the decoder to prioritize contextual information from actions preceding and following the boundary to make the model more robust than relying on unreliable features alone.

Masking for Relation Prior (R)*:* The ordinal relationship represents a core attribute of human behavior. Therefore, during training, we mask the segments of a randomly selected action class $\tilde{c} \in_R \{1,2,\ldots,C\}$ to force the model to deduce the absent action from its adjacent actions. The mask is denoted as $M^{\mathtt{R}}$ where $M_i^{\mathtt{R}} = \mathbb{1}(Y_{0,i,\tilde{c}} \neq 1), i \in \{1,2,\ldots,L\}$. This masked segment modeling benefits the model's ability to leverage inter-relationships of actions.

In summary, our diffusion-based framework seamlessly integrates the three human action priors through the above condition masking strategy. As an interesting interpretation, our model can be viewed to be fully conditional with mask $M^{\mathtt{N}}$, partially conditional with mask $M^{\mathtt{B}}$ and $M^{\mathtt{R}}$, and unconditional with mask $M^{\mathtt{P}}$. This results in a versatile model that comprehensively captures the action distribution from various aspects. Additionally, this interpretation also opens up the potential use of sampling guidance as a means of further improvement, a topic that we will delve into in the subsequent section.

C. Consistency Gradient Guidance

In this section, we introduce a new guidance technique to enhance the conditional information during inference without any retraining (Fig. 3 Bottom). We first provide a description of two established guidance methods, i.e., the classifier guidance and the classifier-free guidance, together with their limitations for our task. Then we motivate and propose our new technique for sampling guidance using a consistency gradient term.

1) Sampling Guidance: Sampling guidance is an alternative way to inject conditional information into an unconditional diffusion model. It can also control the trade-off between the condition strength and the sample diversity of a conditional diffusion model. Classifier guidance [11] and classifier-free guidance [101] are two major methods to achieve sampling guidance. Formally, using the notations in Section III, the classifier guidance injects the gradient of the probability of the target condition from a noise-aware classifier $p_\xi(\mathbb{C}|\mathrm{x}_s)$ into the sampling iteration, and modifies (3) to the following form,

$$\begin{aligned}\mathrm{x}_{s-1} = {} & \sqrt{\bar{\alpha}_{s-1}} f_\theta(\mathrm{x}_s, s) \\ & + \sqrt{1-\bar{\alpha}_{s-1}-\sigma_s^2}\frac{\mathrm{x}_s - \sqrt{\bar{\alpha}_s} f_\theta(\mathrm{x}_s, s)}{\sqrt{1-\bar{\alpha}_s}} + \sigma_s \epsilon \\ & + \gamma \sigma_s^2 \nabla_{\mathrm{x}_s} \log p_\xi(\mathbb{C}|\mathrm{x}_s), \end{aligned} \tag{11}$$

where γ is a scale hyperparameter to control the magnitude of the conditional information. Classifier-free guidance bypasses the need to train a noise-aware classifier, which utilizes the contrast between the outputs from a conditional model and an unconditional model as the guidance direction. At each sampling step, the original model output $f_\theta(\mathrm{x}_s, s, \mathbb{C})$ in (3) is replaced with the following linear combination of conditional and unconditional outputs,

$$\tilde{f}_\theta(\mathrm{x}_s, s, \mathbb{C}) = (1+\gamma) f_\theta(\mathrm{x}_s, s, \mathbb{C}) - \gamma f_\theta(\mathrm{x}_s, s, \varnothing), \tag{12}$$

where γ is the guidance scale and $\varnothing$ is a null condition. The guided model becomes a naive conditional model when $\gamma = 0$ and an unconditional model when $\gamma = -1$.

2) Guidance Using Consistency Gradient: The original purpose of the guidance for diffusion models mentioned above is to trade-off between sampling quality and diversity. This motivation also fits action segmentation, since sampling diversity, unlike generative tasks, is not important in our discriminative task. Therefore, it is desirable to trade off diversity for better quality and enhanced conditional information. However, we find the conventional classifier guidance and classifier-free guidance are not ideal in this case. The classifier guidance necessitates a separate noise-aware classifier which overly complicates the framework, while the classifier-free guidance might bring in temporal discontinuity errors by directly mixing outputs from two model runs. We thus propose a new guidance method, named consistency gradient guidance, which relies on the gradient of

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.

the consistency between the conditional output and the unconditional output.

Specifically, our guidance first obtains two versions of the denoised action sequence P_s at each sampling step, which we denote as $P_s^{\mathtt{N}}$ and $P_s^{\mathtt{P}}$. The first one is the same as in Section IV-A3 with no masking $M^{\mathtt{N}}$ on the encoded features,

$$P_s^{\mathtt{N}} = g_\psi(\hat{Y}_s, s, E \odot M^{\mathtt{N}}), \tag{13}$$

where $P_s^{\mathtt{N}}$ can be regarded as the output from a conditional model. The second one uses the all-zero mask $M^{\mathtt{P}}$,

$$P_s^{\mathtt{P}} = g_\psi(\hat{Y}_s, s, E \odot M^{\mathtt{P}}), \tag{14}$$

where $P_s^{\mathtt{P}}$ can be treated as the output from an unconditional model. We then compute a cross-entropy loss to measure the discrepancy between the two outputs for each frame,

$$\begin{aligned}\mathcal{L}_s^{\text{guide}} = \frac{1}{LC}\sum_{i=1}^{L}\sum_{c=1}^{C}\big[&-P_{s,i,c}^{\mathtt{N}}\log P_{s,i,c}^{\mathtt{P}} \\ &-\left(1-P_{s,i,c}^{\mathtt{N}}\right)\log\left(1-P_{s,i,c}^{\mathtt{P}}\right)\big].\end{aligned} \tag{15}$$

This loss is only used during inference and is computed at each inference step in which we apply the guidance. The negative gradient of this loss $\mathcal{L}_s^{\text{guide}}$ with respect to the current action sequence $\hat{Y}_s$ is used as a guidance term being injected into the sampling iteration. The update rule is thereby modified from (10) to the following one,

$$\begin{aligned}\hat{Y}_{s-1} = &\sqrt{\bar{\alpha}_{s-1}}P_s^{\mathtt{N}} \\ &+\sqrt{1-\bar{\alpha}_{s-1}-\sigma_s^2}\frac{\hat{Y}_s-\sqrt{\bar{\alpha}_s}P_s^{\mathtt{N}}}{\sqrt{1-\bar{\alpha}_s}}+\sigma_s\epsilon \\ &-\gamma\sigma_s^2\nabla_{\hat{Y}_s}\mathcal{L}_s^{\text{guide}},\end{aligned} \tag{16}$$

where γ is a guidance scale. This guidance updates the current action sequence $\hat{Y}_s$ to decrease the cross-entropy loss $\mathcal{L}_s^{\text{guide}}$ and thus increase the consistency between the conditional prediction $P_s^{\mathtt{N}}$ and the unconditional prediction $P_s^{\mathtt{P}}$. The rationale of this guidance is that if the unconditional output $P_s^{\mathtt{P}}$ can be consistent with the conditional output $P_s^{\mathtt{N}}$, the $\hat{Y}_s$ has to reflect enough conditional information. Therefore, the gradient of the consistency represents a direction to enhance the conditional information in $\hat{Y}_s$, which is used to steer the sampling process.

If we compute the gradient $\nabla_{\hat{Y}_s}\mathcal{L}_s^{\text{guide}}$ directly from the guidance loss $\mathcal{L}_s^{\text{guide}}$, the gradient can propagate along the computational graph of both $P_s^{\mathtt{N}}$ and $P_s^{\mathtt{P}}$ as in (15), leading to conflicting gradients in the two opposite directions. Therefore, when computing the gradient, we detach $P_s^{\mathtt{N}}$ with a stop-gradient operation to achieve asymmetric optimization with a clear direction. This is also consistent with our design of the guidance loss $\mathcal{L}_s^{\text{guide}}$ that treats $P_s^{\mathtt{N}}$ as the target outside the logarithmic operator in the cross-entropy in (15) to optimize $P_s^{\mathtt{P}}$ toward $P_s^{\mathtt{N}}$.

Our proposed guidance is related to the classifier guidance in view of similar usage of a gradient term and is also connected to the classifier-free guidance in terms of comparing the outputs from conditional and unconditional models. The merit of this new guidance is to address the previously mentioned limitations of existing guidance methods. Our guidance provides an elegant approach to strengthen the conditional information for action segmentation without using extra modules or directly mixing multiple outputs.

Compared to the vanilla sampling process, the consistency gradient guided sampling requires an additional forward pass of the decoder to obtain $P_s^{\mathtt{P}}$, which incurs computational overhead. In practice, we apply the guidance on steps with intervals instead of all steps to improve efficiency. Specifically, for an accelerated sampling trajectory $\hat{Y}_S, \hat{Y}_{S-\Delta}, \ldots, \hat{Y}_0$ with skips given in Section IV-A3, we only apply the guidance sparsely to the following steps $\hat{Y}_S, \hat{Y}_{S-\zeta\Delta}, \ldots, \hat{Y}_0$. The ζ is a hyperparameter that defines the step interval for the guidance.

V. Experiments

This section introduces the experimental results of the proposed method. We first discuss the setup and implementation details, followed by comparisons with state-of-the-art methods and ablation studies. Finally, the iterative denoising process in our method is examined by qualitative results.

A. Setup

Our method is evaluated on four benchmark datasets, GTEA, 50Salads, Breakfast, and Assembly101, following previous conventions regarding the experimental setup and evaluation metrics.

1) Datasets: Experiments are conducted using four benchmark datasets. *GTEA* [103] comprises 28 egocentric videos capturing daily activities with 11 action classes. On average, each video lasts about one minute and has approximately 19 action instances. *50Salads* [104] includes 50 top-view videos of salad preparation, with 17 classes of actions. The videos in this dataset have an average duration of six minutes, with around 20 action instances in each. *Breakfast* [105] encompasses 1712 third-person view videos and 48 distinct action classes associated with breakfast preparation. These videos typically last about two minutes on average, but the durations vary widely. On average, each video contains seven action instances. *Assembly101* [106] includes 4321 procedural videos with 202 action classes. Each video includes 24 action instances and is 7.1 minutes long on average. Among the four mentioned datasets, Assembly101 is the most expansive in terms of the number of videos and action instances. Consistent with previous studies [4], [5], [7], [47], [53], [56], [57], we use five-fold cross-validation for 50Salads, and four-fold cross-validation for GTEA and Breakfast, adhering to the same data splits as earlier research. For Assembly101, we use the official train-validation split [106] and report results on the validation set following [38].

2) Metrics: In line with previous research, we report the frame-wise accuracy (Acc), the edit score (Edit), and the F1 scores at overlap thresholds 10%, 25%, 50% (F1@{10, 25, 50}). The accuracy metric evaluates the results at the frame level, while the edit score and the F1 scores measure the performance at the segment level. The evaluation scripts are the same as those used in previous studies [4], [7].

3) Implementation Details: For GTEA, 50Salads, and Breakfast, we leverage the I3D features [107] used in most prior works as the input features F with $D = 2048$ dimensions. For Assembly101, following [38], we leverage the TSM features

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON WITH STATE-OF-THE-ART METHODS

Method	GTEA				50Salads				Breakfast			
	F1@{10, 25, 50}	Edit	Acc	Avg	F1@{10, 25, 50}	Edit	Acc	Avg	F1@{10, 25, 50}	Edit	Acc	Avg
[5]MS-TCN++, *TPAMI'20*	88.8 / 85.7 / 76.0	83.5	80.1	82.8	80.7 / 78.5 / 70.1	74.3	83.7	77.5	64.1 / 58.6 / 45.9	65.6	67.6	60.4
[49]MTDA, *WACV'20*	90.5 / 88.4 / 76.2	85.8	80.0	84.2	82.0 / 80.1 / 72.5	75.2	83.2	78.6	74.2 / 68.6 / 56.5	73.6	71.0	68.8
[50]SSTDA, *CVPR'20*	90.0 / 89.1 / 78.0	86.2	79.8	84.6	83.0 / 81.5 / 73.8	75.8	83.2	79.5	75.0 / 69.1 / 55.2	73.7	70.2	68.6
[29]GTRM, *CVPR'20*	- / - / -	-	-	-	75.4 / 72.8 / 63.9	67.5	82.6	72.4	57.5 / 54.0 / 43.3	58.7	65.0	55.7
[6]BCN, *ECCV'20*	88.5 / 87.1 / 77.3	84.4	79.8	83.4	82.3 / 81.3 / 74.0	74.3	84.4	79.3	68.7 / 65.5 / 55.0	66.2	70.4	65.2
[53]ASRF, *WACV'21*	89.4 / 87.8 / 79.8	83.7	77.3	83.6	84.9 / 83.5 / 77.3	79.3	84.5	81.9	74.3 / 68.9 / 56.1	72.4	67.6	67.9
[23]G2L, *CVPR'21*	89.9 / 87.3 / 75.8	84.6	78.5	83.2	80.3 / 78.0 / 69.8	73.4	82.2	76.7	74.9 / 69.0 / 55.2	73.3	70.7	68.6
[45]HASR, *ICCV'21*	90.9 / 88.6 / 76.4	87.5	78.7	84.4	86.6 / 85.7 / 78.5	81.0	83.9	83.1	74.7 / 69.5 / 57.0	71.9	69.4	68.5
[7]ASFormer, *BMVC'21*	90.1 / 88.8 / 79.2	84.6	79.7	84.5	85.1 / 83.4 / 76.0	79.6	85.6	81.9	76.0 / 70.6 / 57.4	75.0	73.5	70.5
[33]TCTr, *IVC'22*	91.3 / 90.1 / 80.0	87.9	81.1	86.1	87.5 / 86.1 / 80.2	83.4	86.6	84.8	76.6 / 71.1 / 58.5	76.1	**77.5**	72.0
[36]FAMMSDTN, *NPL'22*	91.6 / 90.9 / 80.9	88.3	80.7	86.5	86.2 / 84.4 / 77.9	79.9	86.4	83.0	78.5 / 72.9 / 60.2	77.5	74.8	72.8
[25]DPRN, *PR'22*	92.9 / 92.0 / 82.9	90.9	82.0	88.1	87.8 / 86.3 / 79.4	82.0	87.2	84.5	75.6 / 70.5 / 57.6	75.1	71.7	70.1
[55]SEDT, *EL'22*	93.7 / 92.4 / 84.0	91.3	81.3	88.5	89.9 / 88.7 / 81.1	84.7	86.5	86.2	- / - / -	-	-	-
[54]UARL, *IJCAI'22*	92.7 / 91.5 / 82.8	88.1	79.6	86.9	85.3 / 83.5 / 77.8	78.2	84.1	81.8	65.2 / 59.4 / 47.4	66.2	67.8	61.2
[47]BrPrompt†, *CVPR'22*	94.1 / 92.0 / 83.0	91.6	81.2	88.4	89.2 / 87.8 / 81.3	83.8	88.1	86.0	- / - / -	-	-	-
[57]UVAST, *ECCV'22*	92.7 / 91.3 / 81.0	**92.1**	80.2	87.5	89.1 / 87.6 / 81.7	83.9	87.4	85.9	76.9 / 71.5 / 58.0	77.1	69.7	70.6
[102]MCFM‡, *ICIP'22*	91.8 / 91.2 / 80.8	88.0	80.5	86.5	90.6 / 89.5 / 84.2	84.6	90.3	87.8	- / - / -	-	-	-
[56]DTL, *NeurIPS'22*	- / - / -	-	-	-	87.1 / 85.7 / 78.5	80.5	86.9	83.7	78.8 / 74.5 / 62.9	77.7	75.8	73.9
[37]LGTNN, *MS'23*	91.5 / 90.4 / 80.3	87.5	81.2	86.2	87.5 / 86.2 / 79.8	82.0	86.1	84.3	76.2 / 71.5 / 57.5	75.2	72.5	70.6
[34]CETNet, *MTA'23*	91.8 / 91.2 / 81.3	87.9	80.3	86.5	87.6 / 86.5 / 80.1	81.7	86.9	84.6	79.3 / 74.3 / 61.9	77.8	74.9	73.6
[42]C2F-TCN, *TPAMI'23*	92.3 / 90.1 / 80.3	88.5	81.2	86.5	84.3 / 81.7 / 72.8	76.3	84.5	79.9	71.9 / 69.0 / 58.5	68.9	76.6	69.0
[27]SATC, *ICASSP'23*	90.0 / 88.4 / 76.9	84.3	77.9	83.5	85.6 / 84.6 / 79.3	80.2	84.3	82.8	75.8 / 70.8 / 58.6	74.3	70.8	70.1
[60]Patsch *et al.* , *ICASSP'23*	**93.8 / 93.0** / 85.3	91.1	81.7	**89.0**	87.9 / 85.8 / 79.8	81.9	87.4	84.6	73.7 / 71.0 / 61.3	71.7	75.9	70.7
[43]ASPnet†, *CVPR'23*	- / - / -	-	-	-	92.7 / 91.6 / 88.5	91.4	87.5	90.3	78.1 / 72.9 / 60.8	75.9	76.3	72.8
[35]TUT, *ICME'23*	89.0 / 86.4 / 73.3	84.1	76.1	81.8	89.3 / 88.3 / 81.7	84.0	87.2	86.1	76.2 / 71.9 / 60.0	73.7	76.0	71.6
[38]LTContext, *ICCV'23*	- / - / -	-	-	-	89.4 / 87.7 / 82.0	83.2	87.7	86.0	77.6 / 72.6 / 60.1	77.0	74.2	72.3
[52]MVGA, *ICCV'23*	91.3 / 90.5 / 79.3	86.4	80.3	85.6	86.3 / 84.9 / 78.7	81.3	86.0	83.4	75.6 / 72.1 / 59.7	76.8	74.2	71.7
[32]RTK, *ICCV'23*	91.2 / 90.6 / 83.4	87.9	80.3	86.7	87.4 / 86.1 / 79.5	81.4	85.9	84.1	76.9 / 72.4 / 60.5	76.1	73.3	71.8
[63]DFN, *MM'23*	92.3 / 91.7 / 81.4	90.1	81.5	87.4	85.7 / 84.0 / 77.9	81.2	86.9	83.1	78.4 / 74.9 / 63.5	76.8	75.0	73.7
DiffAct, *Ours*	92.5 / 91.5 / 84.7	89.6	82.2	88.1	90.1 / 89.2 / 83.7	85.0	88.9	87.4	80.2 / 76.0 / 64.6	78.1	76.5	75.1
DiffAct++, *Ours*	92.8 / 92.2 / **85.7**	90.0	**82.3**	88.6	**91.0 / 90.1 / 84.9**	**85.8**	**89.1**	**88.2**	**80.4 / 76.3 / 65.1**	**78.3**	76.6	**75.3**

Our method achieves superior results on 50Salads and Breakfast, and comparable performance on GTEA. The average value (Avg) of the five metrics is also presented. †: Extra multimodal features are used. ‡: Extra hand pose features are used.

with $D = 2048$ dimensions. The encoder h_ϕ is a re-implemented ASFormer encoder [7]. The decoder g_ψ is a re-implemented ASFormer decoder modified to be step-dependent, which adds a step embedding to the input as in [12]. Both the encoder and the decoder are single-stage models. The encoder has 10, 10, 12, 12 layers and 64, 64, 256, 256 feature maps for GTEA, 50Salads, Breakfast, and Assembly101 respectively. We adjust the decoder to have fewer layers to reduce the computational cost of the iterative denoising process, which includes 8 layers and 32, 64, 128, 128 feature maps for the four datasets respectively. The intermediate features from encoder layers with indices 5, 7, 9 are concatenated to be the conditioning features E with D'=768 for Breakfast and Assembly101, and D'=192 for other datasets. For the decoder, the concatenation of the conditioning features $E \odot M$ and the noisy sequence Y_s (or $\hat{Y}_s$ at inference) is used as queries and keys in the cross-attention, while the noisy sequence Y_s (or $\hat{Y}_s$ at inference) is taken as values. The step embedding is of 512 dimensions and is added to the values. The encoder and decoder are trained end-to-end using Adam with a batch size of 4. The learning rate is 1e-4 for Breakfast and Assembly101, and 5e-4 for other datasets. In addition to the loss $\mathcal{L}^{\text{sum}}$ for the decoder outputs, we append a prediction head to the encoder and apply $\mathcal{L}^{\text{ce}}$ and $\mathcal{L}^{\text{smo}}$ as auxiliary supervision. The maximum training epochs are 10000, 5000, 1000, 500 for GTEA, 50Salads, Breakfast, and Assembly101 respectively. Following the previous convention [47], we select the best epoch to gather results for *DiffAct*. Concretely, we save the results averaged over all splits every 100 epochs and select the epoch with the best Avg metric. For each dataset, the selected epoch is kept the same for all splits. This epoch selection is applied to all the configurations in ablation studies for *DiffAct* to ensure comparison on results that have equally well converged. For the experiments of *DiffAct++*, we use the same trained model from *DiffAct* but with the guidance term added at the inference stage. All frames are processed together and all actions are predicted together, without any auto-regressive method at training or inference. For the Gaussian smoothing when obtaining the soft ground truth of action boundaries $\bar{B} = \lambda(B)$, the standard deviation of the Gaussian kernel is set as 1, 20, 3, 5 for GTEA, 50Salads, Breakfast, Assembly101 respectively. This is proportional to the different video lengths in different datasets. Our method does not use positional encoding, as it was found harmful in the original ASFormer paper [7].

The total steps are set as $S = 1000$ and 25 steps are utilized at inference based on the sampling strategy with skipped steps [10]. Action sequences are normalized to $[-1, 1]$ when adding and removing noise in (4), (10), and (16). For the consistency gradient guidance, we apply the guidance with a step interval of two ($\zeta = 2$), i.e., applying the guidance once every two steps. The guidance scale γ is set to $2e^4$ on GTEA and 50salads, $2e^2$ on breakfast, and $5e^2$ on Assembly101. On GTEA, we mask the features in (13) for frames whose prediction entropy is higher than the 95% percentile of all frames in this video, since we find that it has negative effects on enhancing the conditional information in ambiguous frames. Our model is implemented with PyTorch and can be trained and tested on a single NVIDIA RTX 2080 GPU.

B. Comparison to State-of-the-Art

Table I presents the experimental results of our method and other recent approaches on GTEA, 50Salads, and Breakfast. We denote our method without consistency gradient guidance as *DiffAct* and the one with guidance as *DiffAct++*. We scrutinize methods in recent years for a thorough comparison and list them

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON WITH STATE-OF-THE-ART METHODS ON ASSEMBLY101

Method	F1@{10, 25, 50}	Edit	Acc	Avg
[5]MS-TCN++, *PAMI'20*	31.6 / 27.8 / 20.6	30.7	37.1	29.6
[7]ASFormer, *BMVC'21*	33.4 / 29.2 / 21.4	30.5	38.8	30.7
[57]UVAST, *ECCV'22*	32.1 / 28.3 / 20.8	31.5	37.4	30.0
[42]C2F-TCN, *TPAMI'23*	33.3 / 29.0 / 21.3	32.4	39.2	31.0
[38]LTContext, *ICCV'23*	33.9 / 30.0 / 22.6	30.4	**41.2**	31.6
DiffAct, *Ours*	35.1 / 32.7 / 26.3	33.4	38.7	33.2
DiffAct++, *Ours*	**35.5 / 33.1 / 26.7**	**34.0**	38.9	**33.6**

We report the results of previous methods given in [38].

TABLE III
ABLATION STUDY ON THE PRIOR MODELING

N	P	B	R	F1@{10, 25, 50}	Edit	Acc	Avg
✓				89.0 / 88.1 / 82.4	83.7	88.1	86.3
✓	✓			89.9 / 88.9 / 82.8	84.3	88.2	86.8
✓		✓		89.7 / 88.6 / 82.6	83.9	88.2	86.6
✓			✓	89.6 / 88.7 / 82.7	84.0	88.0	86.6
✓		✓	✓	89.4 / 88.7 / 83.0	84.4	88.2	86.7
✓	✓		✓	90.0 / 88.8 / 83.4	84.4	88.8	87.1
✓	✓	✓		**90.2 / 89.3** / 83.6	84.6	88.5	87.2
✓	✓	✓	✓	90.1 / 89.2 / **83.7**	**85.0**	**88.9**	**87.4**

N: Baseline. P: Position prior. B: Boundary prior. R: Relation prior.

TABLE IV
ABLATION STUDY ON THE LOSS FUNCTIONS

$\mathcal{L}^{ce}$	$\mathcal{L}^{smo}$	$\mathcal{L}^{bd}$	F1@{10, 25, 50}	Edit	Acc	Avg
✓			86.7 / 85.3 / 79.2	80.8	87.0	83.8
✓	✓		89.8 / 88.9 / 83.1	84.0	88.8	86.9
✓		✓	86.9 / 86.1 / 78.7	81.0	85.4	83.6
✓	✓	✓	**90.1 / 89.2 / 83.7**	**85.0**	**88.9**	**87.4**

TABLE V
ABLATION STUDY ON THE CONDITIONING FEATURES

Features	F1@{10, 25, 50}	Edit	Acc	Avg
Input Features F	82.5 / 80.6 / 72.3	75.7	82.5	78.7
h_ϕ Layer 5	90.3 / 89.2 / **83.9**	**85.1**	**89.1**	**87.5**
h_ϕ Layer 7	**90.4 / 89.4** / 83.4	85.0	88.8	87.4
h_ϕ Layer 9	90.0 / 89.0 / 83.4	84.4	88.8	87.1
h_ϕ Layer 5,7,9	90.1 / 89.2 / 83.7	85.0	88.9	87.4
h_ϕ Prediction	90.3 / 89.3 / 83.3	84.6	87.8	87.1

in ascending order according to the published time. We use the results of previous methods reported in their original papers with default hyper-parameters such as the number of stages. Our proposed method advances the state-of-the-art by an evident margin on 50Salads and Breakfast, and achieves comparable performance on GTEA. Specifically, on 50Salads, the average performance is improved from 86.2 to 87.4 with *DiffAct* and further to 88.2 with *DiffAct++*. On Breakfast, *DiffAct* and *DiffAct++* improve the average performance from 73.9 to 75.1 and 75.3 respectively. On the smallest GTEA dataset, our method obtains a similar overall performance with higher accuracy and F1@50 compared to previous ones. Table II further presents the results on Assembly101, a recent dataset with more lengthy and complex videos. Compared to previous methods, our method offers remarkable gains in terms of segmental metrics, i.e., F1 scores and the edit score. For the accuracy metric, our method achieves comparable results with the ASFormer backbone. Overall, the average performance is evidently improved from 31.6 to 33.2 with *DiffAct* and further to 33.6 with *DiffAct++*. The results in Tables I and II show the effectiveness of our diffusion-based action segmentation as a new framework and its particular advantage on complex datasets. The proposed method can even perform better or on par with previous methods using extra input features such as BrPrompt [47] and ASPnet [43]. Our contribution as a diffusion-based framework for action segmentation is orthogonal to the efforts on the backbone design and representation learning, such as SEDT [55], DPRN [25], and Patsch et al. [60]. Therefore, they can be combined with our framework for further potential improvements.

C. Ablation Studies

We perform comprehensive ablation studies to validate the design decisions in our methodology. The 50Salads dataset is chosen for ablation studies due to its substantial complexity and appropriate data volume. Unless otherwise specified, ablation studies are performed using *DiffAct* to disentangle the effects of the diffusion-based framework itself and the guided sampling process.

Effect of Prior Modeling: To inspect the impact of prior modeling, experiments are conducted in Table III with different combinations of condition masking schemes. For each row in Table III, a masking scheme is randomly selected from the ones ticked in the table at each training iteration for (5). It is observed that our method reaches the best performance when all three priors are considered. Notably, the position prior is especially useful among the three priors, with a decreased result from 87.4 Avg to 86.7 Avg when the position prior is removed. By combining all priors, a remarkable improvement is achieved from 86.3 Avg to 87.4 Avg, validating our condition masking strategy for action prior modeling.

Effect of Training Losses: In Table IV, we investigate the effect of loss functions, where each of the following losses is adopted, the full $\mathcal{L}^{sum}$ loss, the $\mathcal{L}^{sum}$ without $\mathcal{L}^{bd}$, the $\mathcal{L}^{sum}$ without $\mathcal{L}^{smo}$, and the vanilla $\mathcal{L}^{ce}$ loss. It is found that all the loss components are necessary for the best result. On top of $\mathcal{L}^{ce}$ and $\mathcal{L}^{smo}$, our newly proposed boundary alignment loss $\mathcal{L}^{bd}$ brings a slight performance gain in terms of frame-wise accuracy (88.8 versus 88.9 Acc) but an evident gain for temporal continuity (84.0 versus 85.0 Edit). Overall, the average metric is boosted from 86.9 to 87.4 using our proposed boundary alignment loss.

Effect of Conditioning Features: For the generation condition, the intermediate features from the encoder h_ϕ are used in (5). In Table V, we ablate over different choices for the conditioning features, including the input video features F and the features from different layers of the encoder h_ϕ. Performance drops remarkably when using the input feature F as the condition (78.7 Avg versus 87.4 Avg), suggesting the necessity of an encoder. On the other hand, the performance is not sensitive to the encoder layer from which the features are extracted, which ranges from 87.1 Avg to 87.5 Avg.

Effect of Consistency Gradient Guidance: In Table VI, we compare the results with and without our newly proposed consistency gradient guidance, where performance gain is achieved by *DiffAct++* over *DiffAct* (88.2 Avg versus 87.4 Avg). We

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
ABLATION STUDY ON THE SAMPLING GUIDANCE

Method	NFE	F1@{10, 25, 50}	Edit	Acc	Avg
DiffAct	×1	90.1 / 89.2 / 83.7	85.0	88.9	87.4
DiffAct, Steps ×2	×2	90.4 / 89.5 / 84.0	85.3	89.0	87.6
DiffAct, CFG	×2	90.1 / 89.1 / 83.7	84.9	89.0	87.4
DiffAct++	×1.5	**91.0 / 90.1 / 84.9**	**85.8**	**89.1**	**88.2**

CFG: Classifier-free guidance. NFE: Number of function evaluations.

TABLE VII
ABLATION STUDY ON THE GUIDANCE SCALE

Scale	F1@{10, 25, 50}	Edit	Acc	Avg
$2e^2$	90.7 / 89.8 / 84.2	85.4	88.8	87.8
$2e^3$	90.7 / 89.8 / 84.4	85.5	88.9	87.9
$1e^4$	90.9 / 90.0 / 84.7	85.7	**89.1**	88.1
$2e^4$	**91.0 / 90.1 / 84.9**	**85.8**	**89.1**	**88.2**
$3e^4$	90.9 / 90.0 / 84.8	85.7	**89.1**	88.1
$2e^5$	90.2 / 89.2 / 83.9	84.9	88.2	87.3

TABLE VIII
ABLATION STUDY ON THE GUIDANCE INTERVAL

Interval	NFE	F1@{10, 25, 50}	Edit	Acc	Avg
1	×2	**91.0** / 90.0 / **85.0**	**85.9**	**89.1**	**88.2**
2	×1.5	**91.0** / **90.1** / 84.9	85.8	**89.1**	**88.2**
4	×1.25	90.9 / 90.0 / 84.6	85.7	89.0	88.0

also compare the proposed guidance method with two baselines for inference-time enhancement, which are sampling with more steps and sampling with the classifier-free guidance. For classifier-free guidance, we directly use a linear combination of P_s^{N} and P_s^{P} following (12) at each sampling step with a tuned guidance scale. Although the classifier-free guidance can slightly improve the frame-wise accuracy, it damages the edit score due to temporal inconsistency. To take into account the computational overhead, we include the number of function evaluations (NFE) of the decoder in Table VI, which is a commonly used metric to measure the efficiency of diffusion models [108]. Our proposed consistency gradient guidance obtains better results than the two baselines with less overhead.

Effect of Guidance Scales: The guidance scale γ in (16) controls the degree to which the conditional information is enhanced. Table VII reports the results of *DiffAct++* with different guidance scales. It can be seen that the results are stable when the guidance scale falls into a wide range. The performance gain diminishes when the scale approaches zero, and the result becomes inferior when the scale is excessively large.

Effect of Guidance Interval: As mentioned above, we apply our consistency gradient guidance on inference steps with intervals for better efficiency. We test different step intervals ζ and give the results in Table VIII. The selection of this interval is a trade-off between performance and efficiency. Using the guidance on all the sampling steps produces the best performance with a doubled number of decoder evaluations. Applying the guidance with interval two or four yields results similar to all steps but at less computational overhead.

Effect of Inference Steps: Experiment results using different numbers of inference steps for both *DiffAct* and *DiffAct++* are plotted in Fig. 4, from which we can notice a steady increase in all metrics except the accuracy, with diminishing marginal benefits

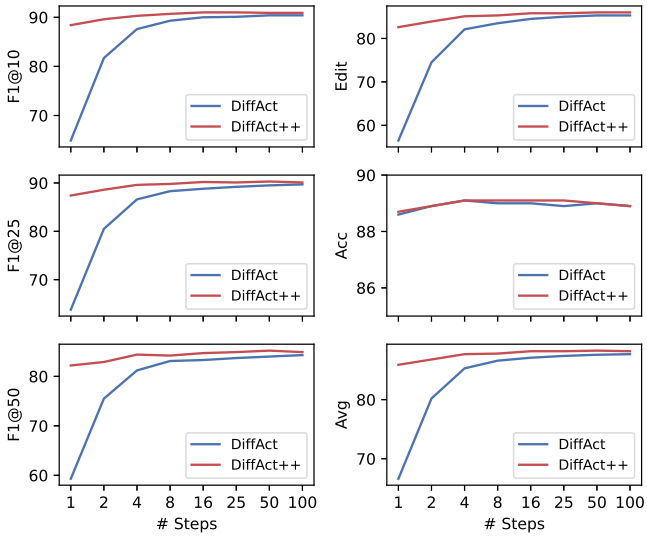


Fig. 4. Ablation study on the number of inference steps.

TABLE IX
RESULTS ON 50SALADS USING TCN-STYLE BACKBONES

Method	F1@{10, 25, 50}	Edit	Acc	Avg
[4]MS-TCN	76.3 / 74.0 / 64.5	67.9	80.7	72.7
[5]MS-TCN++	80.7 / 78.5 / 70.1	74.3	83.7	77.5
[45]HASR (MS-TCN)	83.4 / 81.8 / 71.9	77.4	81.7	79.2
[53]ASRF	84.9 / 83.5 / 77.3	79.3	84.5	81.9
[56]DTL (MS-TCN)	78.3 / 76.5 / 67.6	70.5	81.5	74.9
DiffAct (TCN)	**86.9 / 85.3 / 79.4**	**80.3**	**88.2**	**84.0**

as the number of steps gets larger. The frame-wise accuracy is kept relatively constant across different numbers of steps. This finding where the iterative refinement mainly improves segmental metrics is consistent with the previous findings from multi-stage models, echoing with our motivation on the same underlying mechanism of diffusion models and multi-stage models for action segmentation. The computation grows linearly with the number of steps. We take 25 steps to maintain a good balance between performance and computational cost. Compared to *DiffAct*, we can see that *DiffAct++* achieves better results and faster convergence, indicating a better trade-off between performance and computation when using different numbers of steps.

Effect of the Backbone: The choices of the encoder and decoder in our method are flexible. Therefore, we change our backbone to TCN [4] to show this flexibility. In detail, a single-stage TCN is directly used as the encoder and modified with step embedding to be the decoder. The concatenation of the conditioning features $E \odot M$ and the noisy sequence Y_s (or $\hat{Y}_s$ at inference) is used as the decoder input. The step embedding is added to the input. Table IX compares our results with recent methods using backbones with TCN styles, which show the effectiveness of our method on the different backbone.

D. Stability

This section investigates both the training stability and the testing stability of the proposed method. *Training Stability.* We re-run the main experiment for *DiffAct* on 50Salads ten

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.

TABLE X
TRAINING STABILITY

	F1@{10, 25, 50}	Edit	Acc	Avg
Mean	90.4 / 89.4 / 83.7	84.9	88.5	87.4
Std.	0.11 / 0.12 / 0.18	0.18	0.16	0.15

TABLE XI
INFERENCE STABILITY

	F1@{10, 25, 50}	Edit	Acc	Avg
DiffAct				
Macro Mean	90.3 / 89.4 / 83.9	85.0	88.8	87.5
Macro Std.	0.08 / 0.09 / 0.14	0.16	0.03	0.08
Micro Mean	90.3 / 89.4 / 83.9	85.0	88.7	87.4
Micro Std.	0.86 / 1.03 / 1.25	1.17	0.36	0.93
DiffAct++				
Macro Mean	90.9 / 90.0 / 84.7	85.7	89.0	88.1
Macro Std.	0.09 / 0.08 / 0.13	0.12	0.06	0.08
Micro Mean	91.0 / 90.1 / 84.9	85.8	88.9	88.2
Micro Std.	0.63 / 0.71 / 1.13	0.93	0.43	0.77

TABLE XII
EFFECTS OF VIDEO LENGTH AND ACTION NUMBER

	F1@{10, 25, 50}	Edit	Acc	Avg
DiffAct				
Length Top 50%	88.9 / 88.2 / 82.1	83.4	88.9	86.3
Length Bottom 50%	91.8 / 90.8 / 85.9	87.2	89.0	89.0
#Actions Top 50%	88.3 / 87.0 / 81.6	81.2	88.6	85.3
#Actions Bottom 50%	91.7 / 91.0 / 85.1	87.5	88.7	88.8
DiffAct++				
Length Top 50%	89.6 / 88.8 / 82.8	83.9	88.8	86.8
Length Bottom 50%	92.6 / 91.6 / 87.0	88.0	89.0	89.6
#Actions Top 50%	89.7 / 88.5 / 82.8	82.6	88.9	86.5
#Actions Bottom 50%	92.4 / 91.7 / 86.5	88.7	88.8	89.6

times with different training seeds and the same inference seed for a training stability check. The mean values and standard deviations are reported in Table X, from which we can see that the results are stable with narrow deviations. Note that the consistency gradient guidance of *DiffAct++* only makes modifications at the inference stage and thus has no impact on training stability. *Inference Stability.* Although we fix the random seed during inference to make the results deterministic in our experiments, the sampling process of the diffusion model is by default stochastic. Therefore, we provide the inference stability for both *DiffAct* and *DiffAct++* on 50Salads in Table XI. The inference stability can be measured in two ways, i.e., the macro way and the micro way. The *macro* setting follows the evaluation convention, which repeats the experiment as a whole with ten different seeds at inference. The *micro* setting repeats the inference for each video with ten different seeds and averages the deviations over the videos. The macro result should be used when comparing with the state-of-the-art. Our method is highly stable at inference as in Table XI, and *DiffAct* and *DiffAct++* show a similar level of stability.

E. Impact of Video Length and Number of Actions

To investigate the impact of the length of the video and the number of actions on our proposed method, we report the results on 50Salads in Table XII by dividing the test videos into the top and bottom halves. It can be seen that our method performs well regardless of these factors. Compared to the bottom halves, there are only modest and reasonable performance drops for the top halves given that they are inherently harder samples.

TABLE XIII
THE NUMBER OF PARAMETERS (#PARAMS), THE AVERAGE FLOPS AT INFERENCE (FLOPS), THE GPU MEMORY COST DURING TRAINING (MEM.), THE AVERAGE INFERENCE TIME (TIME), AND THE AVERAGE PERFORMANCE (AVG), MEASURED ON 50SALADS

Method	Avg	#params	FLOPs	Mem.	Time
[5]MS-TCN++	77.5	0.974M	5.73G	1.59G	0.87s
[7]ASFormer	81.9	1.134M	6.66G	3.29G	2.38s
[57]UVAST	85.9	1.035M	3.59G	2.75G	0.75s
DiffAct (4 Steps)	85.3	1.699M	4.21G	1.35G	0.56s
DiffAct (8 Steps)	86.6	1.699M	7.64G	1.35G	0.94s
DiffAct (16 Steps)	87.1	1.699M	14.50G	1.35G	1.72s
DiffAct (25 Steps)	87.4	1.699M	22.22G	1.35G	2.63s
DiffAct++ (4 Steps)	87.7	1.699M	5.93G	1.35G	1.17s
DiffAct++ (8 Steps)	87.8	1.699M	11.07G	1.35G	2.12s
DiffAct++ (16 Steps)	88.2	1.699M	21.36G	1.35G	4.09s
DiffAct++ (25 Steps)	88.2	1.699M	32.51G	1.35G	6.18s

F. Computational Cost

Table XIII compares the computational costs of our method, our backbone ASFormer, and several representative previous methods on 50Salads. When using four steps, our method *DiffAct* largely outperforms ASFormer with fewer FLOPs and time cost at inference and achieves comparable performance at a similar cost compared to UVAST. Using more steps, our method *DiffAct* improves the result with linearly increasing computational overhead, which provides flexible choices balancing between computation and performance to suit different application scenarios. Our method *DiffAct++* further enhances performance, which can be desirable in computation-insensitive scenarios. Another advantage of *DiffAct++* is that it only modifies the inference process without any cost for training or retraining. We anticipate that our method can be further accelerated using advanced diffusion model sampling techniques such as DPM-Solver [108].

G. Qualitative Result

In Fig. 5, we provide a visual illustration of the refinement process throughout the denoising steps using a video from the 50Salads dataset as an example. Starting with a random noise sequence, the model iteratively refines its output until it reaches the final clean action prediction. For example, within the black box in Fig. 5, the 'cut cucumber' segment is initially interrupted by 'cut tomato' and 'peel cucumber' due to their visual similarities. However, as the iterations progress, the model discerns the relationship between these actions, steadily rectifying the errors. As a result, it can successfully predict an uninterrupted 'cut cucumber' segment at the last iteration. Additional outcomes are showcased in Fig. 6, with visualization of the predictions and ground truth sequences from different datasets. Our model displays temporal consistency, precision, and exceptional overall performance.

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.

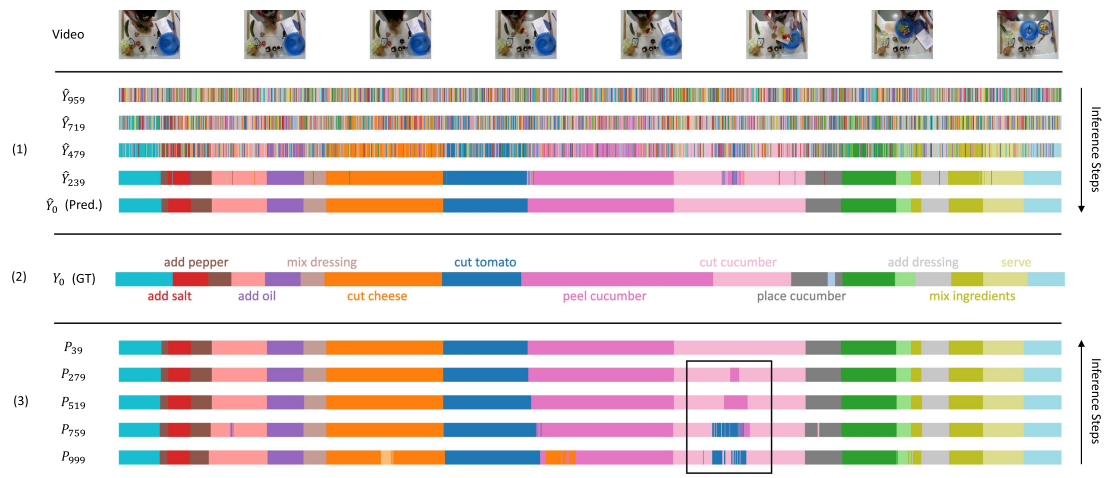


Fig. 5. Visualization of the iterative denoising process. The ground truth is presented in (2), where some segments are marked with class labels. And (1) and (3) respectively plot the inference trajectory $\hat{Y}_s$ and the denoised sequences P_s at different steps (10). Note that P_s is the output of the decoder that is supposed to remove the noise from its input sequence $\hat{Y}_s$, and P_s is then mixed with a gradually reduced level of noise to obtain the intermediate result at the next step $\hat{Y}_{s-\Delta}$. For example, $\hat{Y}_{959}$ is obtained based on P_{999}, and $\hat{Y}_{959}$ is then input to the decoder to acquire P_{959}. In this example, the total inference steps are 25 and 5 steps are visualized for brevity. The video is 'rgb-01-2' from 50Salads.

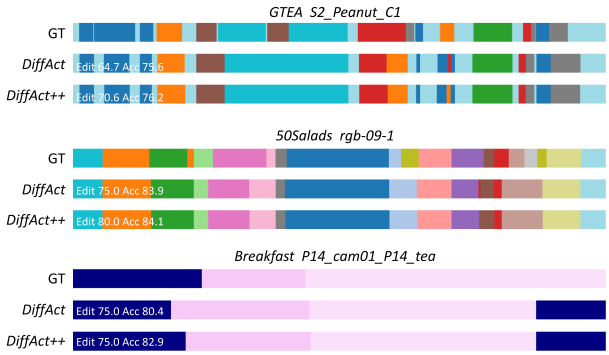


Fig. 6. Visualization of the predictions from our methods. Different datasets use different sets of color codes in the plots.

VI. Discussion

Analysis of the Prior Modeling: In this section, we conduct an exploratory experiment with *DiffAct* on 50Salads to examine how well our model grasps the position prior, the boundary prior and the relation prior. Note that while the default inference process of *DiffAct* uses no masking ($M^{\mathtt{N}}$), in this particular experiment, we employ the masked conditions with each masking type ($M^{\mathtt{P}}, M^{\mathtt{B}}, M^{\mathtt{R}}$) during inference for analysis purposes. Based on the results presented in Table XIV, when mask $M^{\mathtt{B}}$ or mask $M^{\mathtt{R}}$ is used, the performance of the model remains reasonably good, suggesting that it efficiently captures both the boundary and relation priors. It is noteworthy that the outcome under the completely masked condition ($M^{\mathtt{P}}$), exhibiting a 34.8 edit score, significantly outperforms a random guess. This indicates that our position prior modeling has effectively enabled the model to establish links between actions and their temporal positions.

In Fig. 7 (1), we display the condition masks alongside the corresponding action predictions for a video, obtained with each mask type during inference. The model effectively restores the omitted action 'cut tomato' when the condition is masked by $M^{\mathtt{R}}$ at Fig. 7(C) and precisely identifies action boundaries when the condition is masked by $M^{\mathtt{B}}$. Furthermore, when all features are masked by $M^{\mathtt{P}}$, the model generates a generally reasonable action sequence. For example, actions such as cutting and placing ingredients are situated in the center of the timeline (Fig. 7(A)), while actions such as mixing and serving are placed toward the end (Fig. 7(B)). We provide more results using $M^{\mathtt{P}}$ at inference in Fig. 7 (2-4). The generated action sequences notably reflect the characteristics of each dataset. This confirms the efficacy of our model in capturing the underlying distributions of action sequences through generative learning. The above findings illustrate the capability of our method in action prior modeling.

TABLE XIV
Results Using Different Condition Masking Types at the Inference Stage

Masking	F1@{10, 25, 50}	Edit	Acc	Avg
N	90.1 / 89.2 / 83.7	85.0	88.9	87.4
P	25.7 / 21.5 / 11.6	34.8	20.9	22.9
B	89.4 / 88.6 / 83.0	84.1	88.4	86.7
R	88.9 / 87.8 / 81.7	83.5	87.2	85.8

Note that this is only for analysis purposes since the mask types B and R depend on the ground truth. The model performance is maintained at a reasonable level using different masks, suggesting the action priors are well captured.

Discussion with Diffusion Image Segmentation: Diffusion models have also been explored in image segmentation tasks [88], [89], [109]. Our diffusion model tailored for video understanding distinguishes itself from diffusion-based image segmentation. The difference lies in a customized diffusion pipeline, the incorporation of prior modeling, and the introduction of consistency gradient guidance. Specifically, diffusion image segmentation methodologies, such as SegDiff [89], utilize U-Net to predict noise ϵ using a vanilla L2 loss. In contrast, our approach leverages ASFormer and posits that x_0 prediction is

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.

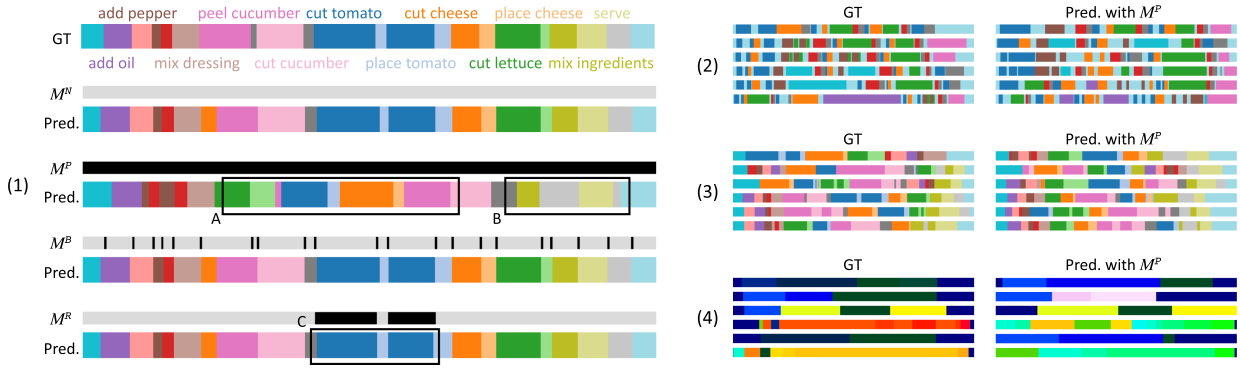


Fig. 7. (1) Visualization of the masks and the corresponding predictions using the masked conditions at inference. The video is 'rgb-03-2' from 50Salads. In M^{N}, M^{P}, M^{B}, M^{R}, masked locations are colored black. (2-4) Ground truth action sequences and predictions using M^{P} at inference for GTEA (2), 50Salads (3), and Breakfast (4). Our model is able to capture the distribution of actions and generate sequences with patterns similar to those of real sequences when all conditions are masked. Different datasets use different sets of color codes in the plots.

more suitable for our objective. Moreover, we combine cross-entropy loss, smoothness loss, and boundary loss to formulate a comprehensive objective for action segmentation. When we naively adapt the diffusion image segmentation method to video data, it yields a significantly lower performance (61.8 Avg) than our method (88.2 Avg) on the 50salads dataset, showing the advantage of our tailored framework for video understanding.

Limitation and Future Work: The relatively expensive inference cost is a well-known shortcoming of diffusion models, which is also present in our diffusion-based models, especially when using a large number of inference steps. This problem can be mitigated through improved sampling solvers [108] or distillation of diffusion models [110]. In this paper, we follow the established evaluation protocol to ensure meaningful comparison with existing methods, which is designed for deterministic methods and requires to fix the inference seed in our generative method to obtain deterministic results. Although we report the worst case, the median case, and the best case results using multiple seeds in the supplementary material, a more comprehensive evaluation protocol tailored for generative methods is worth exploring in future. Another limitation of our proposed method is its relatively smaller advantage on the small-scale GTEA dataset compared to its performance on larger datasets. We hypothesize that the challenge lies in generatively learning the distribution of action sequences with a limited number of videos, resulting in a lower edit score. To address this on small datasets, one potential solution is to substitute the Gaussian noise in the diffusion process with perturbations grounded in action statistics. For example, corrupting data points towards the mean sequence derived from the training set might reduce the hypothesis space, thereby reducing the data demands. The generation of frame-wise action sequences and segment-wise ordered action lists could also be combined in future studies in our diffusion-based action segmentation. Additionally, leveraging our generative framework for unified action segmentation and anticipation appears promising, given its inherent suitability for anticipation with prior modeling. Another potential future direction is to generalize our method to more challenging settings of action segmentation, such as multi-view and multi-label action segmentation, by replacing the one-hot label sequences with multi-hot label sequences and incorporating cross-modality fusion in the condition of the diffusion model. The method design should also address the discrepancy between the data characteristics of multi-view multi-label datasets (e.g., TSU [67] and Charades [68]) and the datasets used in this study.

VII. Conclusion

This study proposes a new framework for temporal action segmentation that generates action sequences through an iterative denoising procedure. A flexible condition masking strategy is developed to harness the positional, boundary, and relational priors of human activities. Furthermore, we suggest a consistency gradient guidance technique to enhance the conditional information during the inference process. Our framework achieves state-of-the-art results on multiple datasets, paving the way for a promising direction for action segmentation.

Acknowledgment

The authors acknowledge the National Computational Infrastructure of Australia for providing computational resources via the NCI AI Flagship Scheme and the Sydney NCI Scheme. We also acknowledge the High-Performance Computing Platform of Peking University for providing computational resources.

References

[1] S. Vishwakarma and A. Agrawal, "A survey on activity recognition and behavior understanding in video surveillance," *Vis. Comput.*, vol. 29, pp. 983–1009, 2013.
[2] E. Apostolidis, E. Adamantidou, A. I. Metsai, V. Mezaris, and I. Patras, "Video summarization using deep neural networks: A survey," in *Proc. IEEE*, vol. 109, no. 11, pp. 1838–1863, Nov. 2021.
[3] D. Liu et al., "Towards unified surgical skill assessment," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 9522–9531.
[4] Y. A. Farha and J. Gall, "MS-TCN: Multi-stage temporal convolutional network for action segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 3575–3584.
[5] S.-J. Li, Y. AbuFarha, Y. Liu, M.-M. Cheng, and J. Gall, "MS-TCN++: Multi-stage temporal convolutional network for action segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 6, pp. 6647–6658, Jun. 2023.
[6] Z. Wang, Z. Gao, L. Wang, Z. Li, and G. Wu, "Boundary-aware cascade networks for temporal action segmentation," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2020, pp. 34–51.

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.

[7] F. Yi, H. Wen, and T. Jiang, "ASFormer: Transformer for action segmentation," in *Proc. Brit. Mach. Vis. Conf.*, 2021, Art. no. 236.

[8] G. Ding, F. Sener, and A. Yao, "Temporal action segmentation: An analysis of modern technique," 2022, *arXiv:2210.10352*.

[9] F.-A. Croitoru, V. Hondru, R. T. Ionescu, and M. Shah, "Diffusion models in vision: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 9, pp. 10850–10869, Sep. 2023.

[10] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–12.

[11] P. Dhariwal and A. Nichol, "Diffusion models beat GANs on image synthesis," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 8780–8794.

[12] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 6840–6851.

[13] D. Liu, Q. Li, A.-D. Dinh, T. Jiang, M. Shah, and C. Xu, "Diffusion action segmentation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 10 139–10 149.

[14] L. Ding and C. Xu, "TricorNet: A hybrid temporal convolutional and recurrent network for video action segmentation," 2017, *arXiv: 1705.07818*.

[15] B. Singh, T. K. Marks, M. Jones, O. Tuzel, and M. Shao, "A multi-stream bi-directional recurrent neural network for fine-grained action detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 1961–1970.

[16] C. Lea, A. Reiter, R. Vidal, and G. D. Hager, "Segmental spatiotemporal CNNs for fine-grained action segmentation," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2016, pp. 36–52.

[17] C. Lea, M. D. Flynn, R. Vidal, A. Reiter, and G. D. Hager, "Temporal convolutional networks for action segmentation and detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 156–165.

[18] P. Lei and S. Todorovic, "Temporal deformable residual networks for action segmentation in videos," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 6742–6751.

[19] K.-N. C. Mac, D. Joshi, R. A. Yeh, J. Xiong, R. S. Feris, and M. N. Do, "Learning motion in feature space: Locally-consistent deformable convolution networks for fine-grained action detection," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 6282–6291.

[20] J. Wang, Z. Du, A. Li, and Y. Wang, "Atrous temporal convolutional network for video action segmentation," in *Proc. IEEE Int. Conf. Image Process.*, 2019, pp. 1585–1589.

[21] B. Hampiholi, C. Jarvers, W. Mader, and H. Neumann, "Depthwise separable temporal convolutional network for action segmentation," in *Proc. Int. Conf. 3D Vis.*, 2020, pp. 633–641.

[22] Y. Souri, Y. A. Farha, F. Despinoy, G. Francesca, and J. Gall, "FIFA: Fast inference approximation for action segmentation," in *Proc. DAGM German Conf. Pattern Recognit.*, Springer, 2021, pp. 282–296.

[23] S.-H. Gao, Q. Han, Z.-Y. Li, P. Peng, L. Wang, and M.-M. Cheng, "Global2local: Efficient structure search for video action segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 16 805–16 814.

[24] Y. Li et al., "Efficient two-step networks for temporal action segmentation," *Neurocomputing*, vol. 454, pp. 373–381, 2021.

[25] J. Park, D. Kim, S. Huh, and S. Jo, "Maximization and restoration: Action segmentation through dilation passing and temporal reconstruction," *Pattern Recognit.*, vol. 129, 2022, Art. no. 108764.

[26] Y. Zhang, K. Ren, C. Zhang, and T. Yan, "SG-TCN: Semantic guidance temporal convolutional network for action segmentation," in *Proc. Int. Joint Conf. Neural Netw.*, 2022, pp. 1–8.

[27] L. Yang, Y. Jiang, J. Hong, Z. Wu, Z. Yang, and J. Long, "Stacking-based attention temporal convolutional network for action segmentation," in *Proc. Int. Conf. Acoust. Speech Signal Process.*, 2023, pp. 1–5.

[28] K. Nakamura et al., "Regularizing oversmoothing of temporal convolutional networks for action segmentation into human assembly operations," *J. Signal Process.*, vol. 27, no. 4, pp. 75–79, 2023.

[29] Y. Huang, Y. Sugano, and Y. Sato, "Improving action segmentation via graph-based temporal reasoning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 14 024–14 034.

[30] J. Zhang, P.-H. Tsai, and M.-H. Tsai, "Semantic2Graph: Graph-based multi-modal feature for action segmentation in videos," 2022, *arXiv:2209.05653*.

[31] Y.-H. Li, K.-Y. Liu, S.-L. Liu, L. Feng, and H. Qiao, "Involving distinguished temporal graph convolutional networks for skeleton-based temporal action segmentation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 34, no. 1, pp. 647–660, Jan. 2024.

[32] B. Jiang, Y. Jin, Z. Tan, and Y. Mu, "Video action segmentation via contextually refined temporal keypoints," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 13 836–13 845.

[33] N. Aziere and S. Todorovic, "Multistage temporal convolution transformer for action segmentation," *Image Vis. Comput.*, vol. 128, 2022, Art. no. 104567.

[34] J. Wang, Z. Wang, S. Zhuang, Y. Hao, and H. Wang, "Cross-enhancement transformer for action segmentation," *Multimedia Tools Appl.*, vol. 83, pp. 25643–25656, 2024.

[35] D. Du, B. Su, Y. Li, Z. Qi, L. Si, and Y. Shan, "Do we really need temporal convolutions in action segmentation?," in *Proc. IEEE Int. Conf. Multimedia Expo*, 2023, pp. 1014–1019.

[36] Z. Du and Q. Wang, "Dilated transformer with feature aggregation module for action segmentation," *Neural Process. Lett.*, vol. 55, pp. 6181–6197, 2023.

[37] X. Tian, Y. Jin, and X. Tang, "Local-Global transformer neural network for temporal action segmentation," *Multimedia Syst.*, vol. 29, no. 2, pp. 615–626, 2023.

[38] E. Bahrami, G. Francesca, and J. Gall, "How much temporal long-term context is needed for action segmentation?," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 10 351–10 361.

[39] X. Tian, Y. Jin, and X. Tang, "TSRN: Two-stage refinement network for temporal action segmentation," *Pattern Anal. Appl.*, vol. 26, pp. 1375–1393, 2023.

[40] S. Reza, B. Sundareshan, M. Moghaddam, and O. Camps, "Enhancing transformer backbone for egocentric video action segmentation," 2023, *arXiv:2305.11365*.

[41] D. Wang, Y. Yuan, and Q. Wang, "Gated forward refinement network for action segmentation," *Neurocomputing*, vol. 407, pp. 63–71, 2020.

[42] D. Singhania, R. Rahaman, and A. Yao, "C2F-TCN: A framework for semi-and fully-supervised temporal action segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 10, pp. 11484–11501, Oct. 2023.

[43] B. van Amsterdam, A. Kadkhodamohammadi, I. Luengo, and D. Stoyanov, "ASPnet: Action segmentation with shared-private representation of multiple data sources," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 2384–2393.

[44] S. Yan et al., "UnLoc: A unified framework for video localization tasks," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 13 623–13 633.

[45] H. Ahn and D. Lee, "Refining action segmentation with hierarchical video representations," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 16 302–16 310.

[46] F. Sener, D. Singhania, and A. Yao, "Temporal aggregate representations for long-range video understanding," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2020, pp. 154–171.

[47] M. Li et al., "Bridge-Prompt: Towards ordinal action understanding in instructional videos," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 19 880–19 889.

[48] E. Dessalene, M. Maynord, C. Fermüller, and Y. Aloimonos, "Therbligs in action: Video understanding through motion primitives," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 10 618–10 626.

[49] M.-H. Chen, B. Li, Y. Bao, and G. AlRegib, "Action segmentation with mixed temporal domain adaptation," in *Proc. Winter Conf. Appl. Comput. Vis.*, 2020, pp. 605–614.

[50] M.-H. Chen, B. Li, Y. Bao, G. AlRegib, and Z. Kira, "Action segmentation with joint self-supervised temporal domain adaptation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9454–9463.

[51] H. Han, Z. Lu, and J.-R. Wen, "CTDA: Contrastive temporal domain adaptation for action segmentation," in *Proc. Int. Conf. Multimedia Model.*, Springer, 2023, pp. 562–574.

[52] N. Aziere and S. Todorovic, "Markov game video augmentation for action segmentation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 13 505–13 514.

[53] Y. Ishikawa, S. Kasai, Y. Aoki, and H. Kataoka, "Alleviating over-segmentation errors by detecting action boundaries," in *Proc. Winter Conf. Appl. Comput. Vis.*, 2021, pp. 2322–2331.

[54] L. Chen, M. Li, Y. Duan, J. Zhou, and J. Lu, "Uncertainty-aware representation learning for action segmentation," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, Art. no. 6.

[55] G.-H. Kim and E. Kim, "Stacked encoder-decoder transformer with boundary smoothing for action segmentation," *Electron. Lett.*, vol. 58, no. 25, pp. 972–974, 2022.

[56] Z. Xu, Y. S. Rawat, Y. Wong, M. Kankanhalli, and M. Shah, "Don't pour cereal into coffee: Differentiable temporal logic for temporal action segmentation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 14 890–14 903.

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.

[57] N. Behrmann, S. A. Golestaneh, Z. Kolter, J. Gall, and M. Noroozi, "Unified fully and timestamp supervised temporal action segmentation via sequence to sequence translation," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2022, pp. 52–68.

[58] Z. Liu et al., "Temporal segment transformer for action segmentation," 2023, *arXiv:2302.13074*.

[59] P. Wang and H. Ling, "DIR-AS: Decoupling individual identification and temporal reasoning for action segmentation," 2023, *arXiv:2304.02110*.

[60] C. Patsch and E. Steinbach, "Self-attention based action segmentation using intra-and inter-segment representations," in *Proc. Int. Conf. Acoust. Speech Signal Process.*, 2023, pp. 1–5.

[61] Z. Lu and E. Elhamifar, "BIT: Bi-level temporal modeling for efficient supervised action segmentation," 2023, *arXiv:2308.14900*.

[62] H. Gammulle, T. Fernando, S. Denman, S. Sridharan, and C. Fookes, "Coupled generative adversarial network for continuous fine-grained action segmentation," in *Proc. Winter Conf. Appl. Comput. Vis.*, 2019, pp. 200–209.

[63] B. Jiang and Y. Mu, "Diffused fourier network for video action segmentation," in *Proc. 31st ACM Int. Conf. Multimedia*, 2023, pp. 5474–5483.

[64] A. Piergiovanni and M. Ryoo, "Temporal Gaussian mixture layer for videos," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 5152–5161.

[65] D. Yang et al., "LAC: Latent action composition for Skeleton-based action segmentation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 13633–13644.

[66] R. Dai, S. Das, K. Kahatapitiya, M. S. Ryoo, and F. Brémond, "MS-TCT: Multi-scale temporal ConvTransformer for action detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 20009–20019.

[67] R. Dai et al., "Toyota smarthome untrimmed: Real-world untrimmed videos for activity detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 2, pp. 2533–2550, Feb. 2023.

[68] G. A. Sigurdsson, G. Varol, X. Wang, A. Farhadi, I. Laptev, and A. Gupta, "Hollywood in homes: Crowdsourcing data collection for activity understanding," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 510–526.

[69] A. Richard, H. Kuehne, A. Iqbal, and J. Gall, "NeuralNetwork-Viterbi: A framework for weakly supervised video learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 7386–7395.

[70] G. Ding and A. Yao, "Leveraging action affinity and continuity for semi-supervised temporal action segmentation," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 17–32.

[71] Z. Du, X. Wang, G. Zhou, and Q. Wang, "Fast and unsupervised action boundary detection for action segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 3313–3322.

[72] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep unsupervised learning using nonequilibrium thermodynamics," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 2256–2265.

[73] Y. Song and S. Ermon, "Generative modeling by estimating gradients of the data distribution," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, Art. no. 1067.

[74] Y. Song and S. Ermon, "Improved techniques for training score-based generative models," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 12 438–12 448.

[75] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," 2020, *arXiv: 2011.13456*.

[76] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10 684–10 695.

[77] A. K. Bhunia et al., "Person image synthesis via denoising diffusion model," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 5968–5976.

[78] P. Yu et al., "Latent diffusion energy-based model for interpretable text modeling," 2022, *arXiv:2206.05895*.

[79] S. Gu et al., "Vector quantized diffusion model for text-to-image synthesis," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10 696–10 706.

[80] G. Kim and J. C. Ye, "DiffusionClip: Text-guided image manipulation using diffusion models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 2416–2425.

[81] Y. Leng et al., "BinauralGrad: A two-stage conditional diffusion probabilistic model for binaural audio synthesis," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 23 689–23 700.

[82] M. W. Lam, J. Wang, D. Su, and D. Yu, "BDDM: Bilateral denoising diffusion models for fast and high-quality speech synthesis," 2022, *arXiv:2203.13508*.

[83] R. Yang, P. Srivastava, and S. Mandt, "Diffusion probabilistic modeling for video generation," 2022, *arXiv:2203.09481*.

[84] T. Höppe, A. Mehrjou, S. Bauer, D. Nielsen, and A. Dittadi, "Diffusion models for video prediction and infilling," 2022, *arXiv:2206.07696*.

[85] V. Voleti, A. Jolicoeur-Martineau, and C. Pal, "MCVD-masked conditional video diffusion for prediction, generation, and interpolation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 23 371–23 385.

[86] S. Ge et al., "Preserve your own correlation: A noise prior for video diffusion models," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 22873–22884.

[87] S. Chen, P. Sun, Y. Song, and P. Luo, "DiffusionDet: Diffusion model for object detection," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 19773–19786.

[88] D. Baranchuk, I. Rubachev, A. Voynov, V. Khrulkov, and A. Babenko, "Label-efficient semantic segmentation with diffusion models," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–11.

[89] T. Amit, E. Nachmani, T. Shaharbany, and L. Wolf, "SegDiff: Image segmentation with diffusion probabilistic models," 2021, *arXiv:2112.00390*.

[90] J. Gong, L. G. Foo, Z. Fan, Q. Ke, H. Rahmani, and J. Liu, "DiffPose: Toward more reliable 3D pose estimation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 13 041–13 051.

[91] R. Feng, Y. Gao, T. H. E. Tse, X. Ma, and H. J. Chang, "DiffPose: SpatioTemporal diffusion model for video-based human pose estimation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 14815–14826.

[92] L. Sweeney, G. Healy, and A. F. Smeaton, "Diffusing surrogate dreams of video scenes to predict video memorability," 2022, *arXiv:2212.09308*.

[93] X. Zhong, Z. Li, S. Chen, K. Jiang, C. Chen, and M. Ye, "Refined semantic enhancement towards frequency diffusion for video captioning," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 3724–3732.

[94] S. Nag, X. Zhu, J. Deng, Y.-Z. Song, and T. Xiang, "DiffTAD: Temporal action detection with proposal denoising diffusion," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 10328–10340.

[95] H. Zhao, K. Q. Lin, R. Yan, and Z. Li, "DiffusionVMR: Diffusion model for video moment retrieval," 2023, *arXiv:2308.15109*.

[96] A. Flaborea, L. Collorone, G. D'Amely, S. D'Arrigo, B. Prenkaj, and F. Galasso, "Multimodal motion conditioned diffusion model for skeleton-based video anomaly detection," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 10284–10295.

[97] A. Awasthi et al., "Anomaly detection in satellite videos using diffusion models," 2023, *arXiv:2306.05376*.

[98] A. O. Tur, N. Dall'Asen, C. Beyan, and E. Ricci, "Exploring diffusion models for unsupervised video anomaly detection," in *Proc. IEEE Int. Conf. Image Process.*, 2023, pp. 2540–2544.

[99] C. Luo, "Understanding diffusion models: A unified perspective," 2022, *arXiv:2208.11970*.

[100] K. Preechakul, N. Chatthee, S. Wizadwongsa, and S. Suwajanakorn, "Diffusion autoencoders: Toward a meaningful and decodable representation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10 619–10 629.

[101] J. Ho and T. Salimans, "Classifier-free diffusion guidance," in *Proc. NeurIPS 2021 Workshop Deep Generative Models Downstream Appl.*, 2021, pp. 1–8.

[102] K. Ishihara, G. Nakano, and T. Inoshita, "MCFM: Mutual cross fusion module for intermediate fusion-based action segmentation," in *Proc. IEEE Int. Conf. Image Process.*, 2022, pp. 1701–1705.

[103] A. Fathi, X. Ren, and J. M. Rehg, "Learning to recognize objects in egocentric activities," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2011, pp. 3281–3288.

[104] S. Stein and S. J. McKenna, "Combining embedded accelerometers with computer vision for recognizing food preparation activities," in *Proc. 2013 ACM Int. Joint Conf. Pervasive Ubiquitous Comput.*, 2013, pp. 729–738.

[105] H. Kuehne, A. Arslan, and T. Serre, "The language of actions: Recovering the syntax and semantics of goal-directed human activities," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2014, pp. 780–787.

[106] F. Sener et al., "Assembly101: A large-scale multi-view video dataset for understanding procedural activities," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 21 096–21 106.

[107] J. Carreira and A. Zisserman, "Quo vadis, action recognition? A new model and the Kinetics dataset," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 6299–6308.

[108] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu, "DPM-Solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 5775–5787.

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.

[109] J. Wu, H. Fang, Y. Zhang, Y. Yang, and Y. Xu, "MedSegDiff: Medical image segmentation with diffusion probabilistic model," 2022, *arXiv:2211.00611*.
[110] T. Salimans and J. Ho, "Progressive distillation for fast sampling of diffusion models," 2022, *arXiv:2202.00512*.



Daochang Liu received the BE degree from Tongji University, in 2017, and the PhD degree from Peking University, in 2022. He is a lecturer with the School of Physics, Mathematics and Computing, University of Western Australia. He was a postdoctoral researcher with the University of Sydney. His research interests include rooted in understanding human action and skills with computer vision, which also involves fundamental problems in generative models and applications in healthcare.



Qiyue Li received the bachelor's degree from Peking University, where she is currently working toward the PhD degree. Her research is primarily concentrating on video action understanding and analysis with generative and transformer-based methods in the field of computer vision. She is also interested in multimodal information processing, which includes action quality assessment and video object segmentation, particularly within the context of medical applications.



Anh-Dung Dinh received the BEngineer and MPhil degrees from the Hanoi University of Science and Technology, in 2019 and 2022. He is currently working toward the PhD degree with the University of Sydney, embarking on his academic journey in 2022. His research interests lie in explaining machine learning and deep learning models with specific applications in computer visions. His recent research contributions have been dedicated to image generative models.



Tingting Jiang received the BS degree in computer science from the University of Science and Technology of China, Hefei, China, in 2001, and the PhD degree in computer science from Duke University, USA, in 2007. She is now an associate professor with the School of Computer Science, Peking University, Beijing, China. Her research interests include computer vision, image and video quality assessment, biological and medical image processing.



Mubarak Shah (Fellow, IEEE) the UCF trustee chair professor, is the founding director of Center for Research in Computer Visions, University of Central Florida (UCF). He has published extensively on topics related to human activity and action recognition, visual tracking, geo localization, visual crowd analysis, object detection and categorization, shape from shading, etc. He has served as ACM and IEEE distinguished visitor program speaker. He is a fellow of the ACM, AAAS, NAI, IAPR, AAIA, and SPIE. He is a recipient of 2024 Undergraduate Research Faculty Mentoring Award from the Computing Research Association Committee on Education (CRA-E); 2022 PAMI Mark Everingham Prize for pioneering human action recognition datasets; 2020 ACM SIGMM Test of Time Honorable Mention Award for his paper "Visual attention detection in video sequences using spatiotemporal cues"; 2020 International Conference on Pattern Recognition (ICPR) Best Scientific Paper Award; 2019 ACM SIGMM Technical Achievement award; an honorable mention for the ICCV 2005 Where Am I? Challenge Problem; 2013 NGA Best Research Poster Presentation; 2nd place in Grand Challenge at the ACM Multimedia 2013 conference; and runner up for the best paper award in ACM Multimedia Conference in 2005 and 2010.



Chang Xu (Senior Member, IEEE) is ARC future fellow and associate professor with the School of Computer Science, University of Sydney. He received the NSW Premier's Prize for Early Career Researcher of the Year and the University of Sydney Vice-Chancellor's Award for Outstanding Early Career Research. His research interests lie in machine learning algorithms and related applications in computer vision. He has published more than 100 papers in prestigious journals and top tier conferences. He has received several paper awards, including Distinguished Paper Award in AAAI 2023, Best Student Paper Award in ICDM 2022, Best Paper Candidate in CVPR 2021, and Distinguished Paper Award in IJCAI 2018. He served as an area chair of NeurIPS, ICML, ICLR, KDD, CVPR, and MM, as well as a senior PC member of AAAI and IJCAI. In addition, he served as an associate editor of *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *IEEE Transactions on Multimedia*, and *Transactions on Machine Learning Research*. He has been named a Top Ten distinguished senior PC member in IJCAI 2017 and an outstanding associate editor of the *IEEE Transactions on Multimedia* in 2022.

Authorized licensed use limited to: New York University. Downloaded on October 14,2026 at 23:59:35 UTC from IEEE Xplore. Restrictions apply.